

Terrain modeling and TIN

Source : Dr. Michela Bertolotto

Terrain data

Terrain data relates to the 3D configuration of the surface of the Earth

On the other hand, ***map data*** refers to data located on the surface of the Earth (2D)

The geometry of a terrain is modeled as a $2\frac{1}{2}$ -dimensional surface, i.e., a surface in 3D space described by a bivariate function

Mathematical terrain models

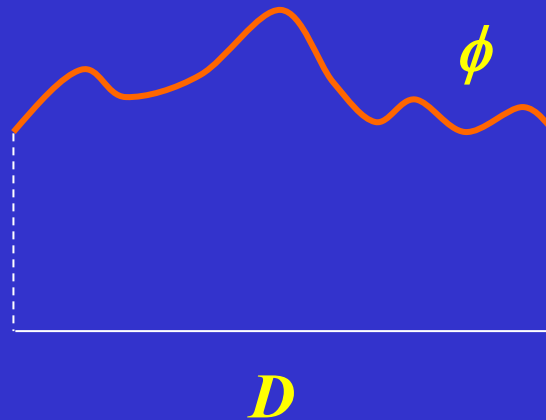
A *topographic surface* or *terrain* can be mathematically modeled by the image of a real bivariate function

$$z = \phi(x, y)$$

defined over a domain D such that $D \subseteq \mathbb{R}^2$

The pair $T=(D, \phi)$ is called a *mathematical terrain model*

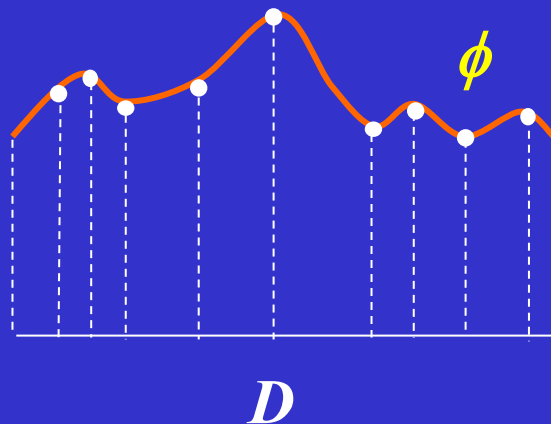
Unidimensional
profile of a
*mathematical
terrain model*



Digital Terrain Models (DTM)

A *digital terrain model* is a model providing a representation of a terrain on the basis of a finite set of sampled data

Elevation data refers to measures of elevation at a set of points V of the domain plus possibly a set E of non-crossing line segments with endpoints in V



Elevation data acquisition

Elevation data can be acquired through:

- *sampling technologies* (by means of on-site measurements or of remote sensing techniques)
- *digitisation* of existing contour maps

Elevation data can be scattered (irregularly distributed) or form a regular grid

The set of non-crossing lines can form a collection of polygonal chains

Contours

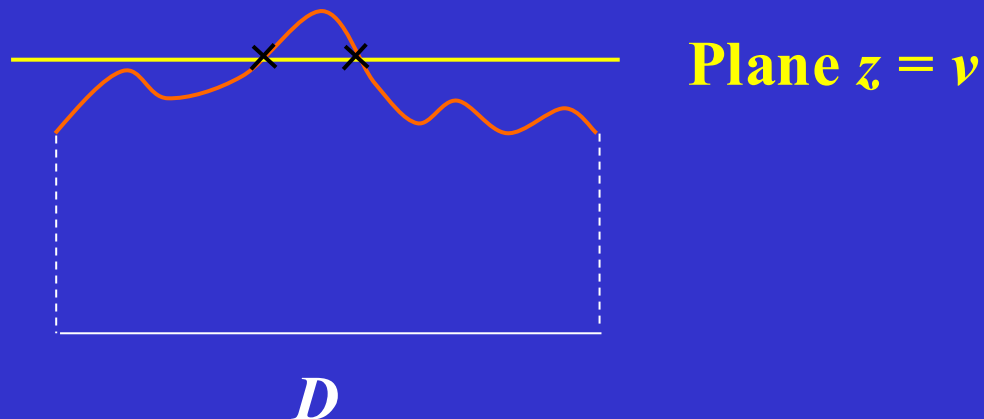
Given a *terrain model*

$$T = (D, \phi)$$

and a real value v , the set of *contours* of T at height v is

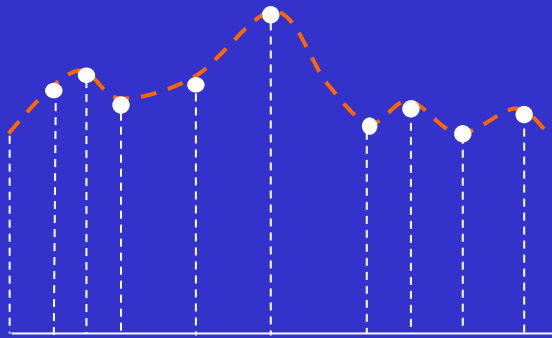
$$\{ (x,y) \in D, \phi(x,y) = v \}$$

This is a set of simple lines (non self-intersecting)

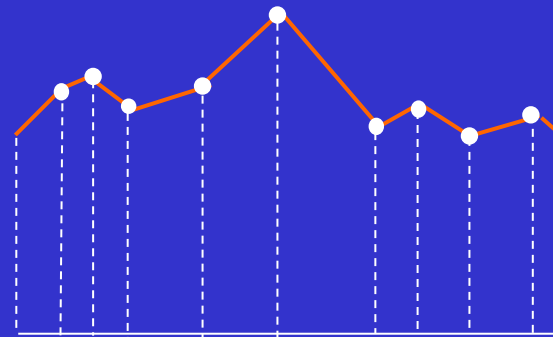


DTMs

Digital terrain models represent an approximation of mathematical terrain models



Sampled model



Digital terrain model

Sampled data distribution

Sampled data can be scattered (irregularly distributed) or form a regular grid on the domain

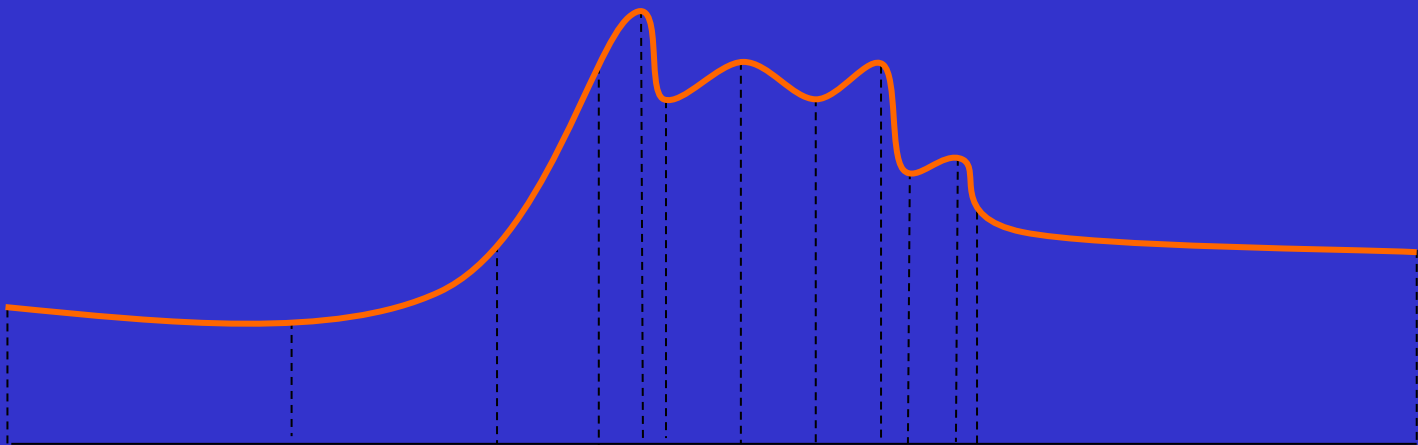
The distribution of the sampled data can depend on the acquisition technique or on the specific application

Different distributions might be required by different configurations of the terrain relief

Sampled data distribution

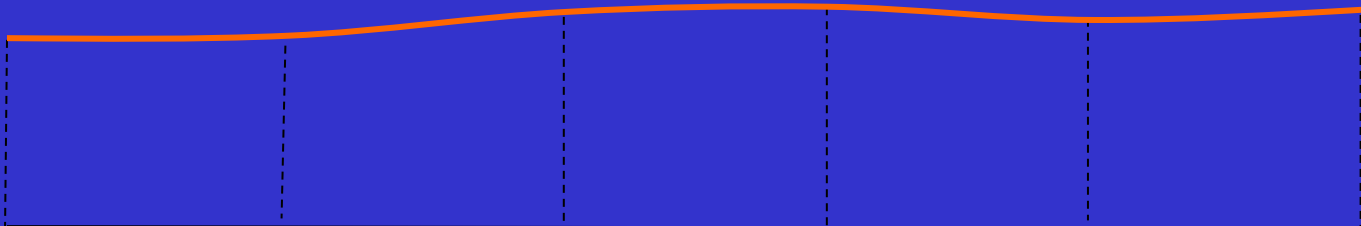
Sometimes it can be useful to have *irregularly distributed* sets of data

For example, only a few sampled points where the terrain is quite flat and more values where the surface presents specific features such as peaks etc.

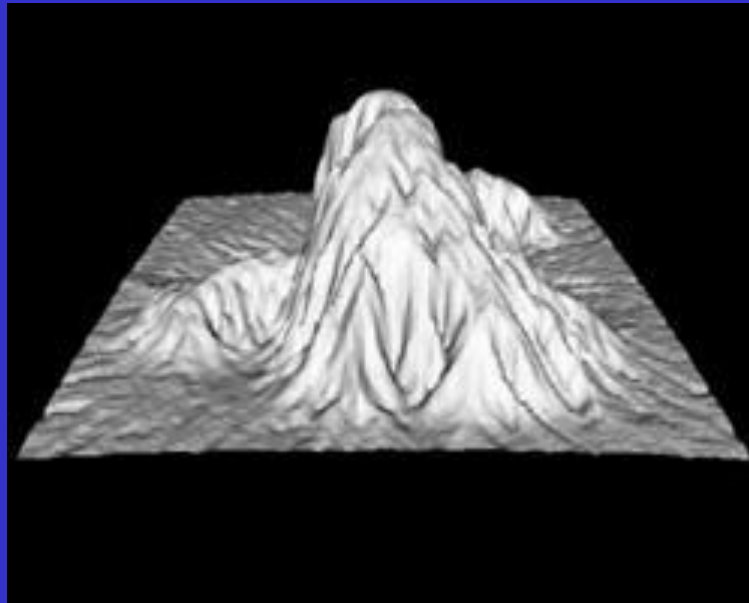
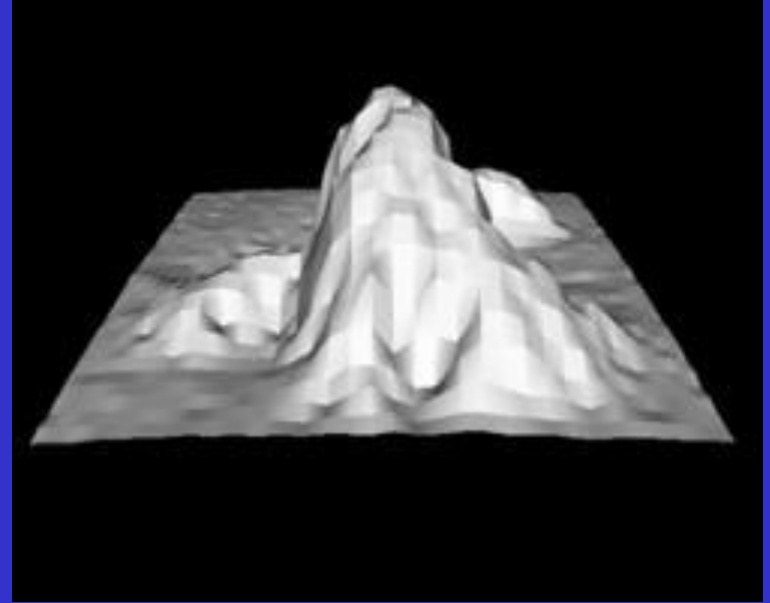
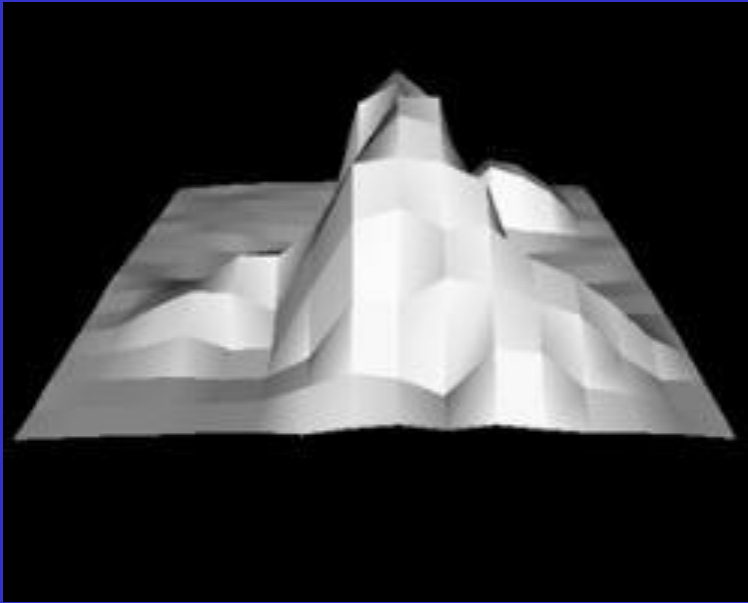


Sampled data distribution (cont.d)

Regular sampling is good in areas where the terrain elevation is more or less constant



DTMs



In general, a larger number of sampled points allows for a better representation:

Terrain models

- **Global** terrain models: defined by means of a single function interpolating all data

- **Local** terrain models: piecewise defined on a partition of the domain into “patches” (regions)

In other words, they represent the terrain by means of a different function on each of the regions in which the domain is subdivided

- In general it is very difficult to find a single function that interpolates all available data, so usually local models are used

Types of DTMs

- *Polyhedral terrain models*
- *Gridded elevation models*
- *Contour maps*

Polyhedral terrain models: definition

A polyhedral terrain model for a set of sampled points V can be defined on the basis of:

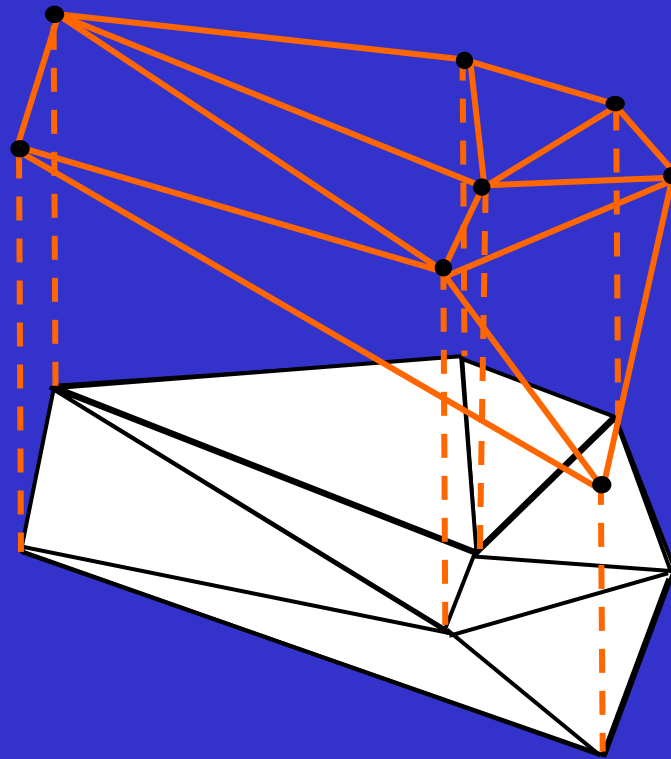
1. a **partition** of the domain D into polygonal regions having their vertices at points in V
2. a **function** f that is *linear* over each region of the partition (i.e., the image of f over each polygonal region is a planar patch – this will guarantee continuity of the surface along the common edges) (f is also called a *piecewise linear* function). Image analysis techniques called *facet models* have some similarity with these methods.

Polyhedral terrain models: properties

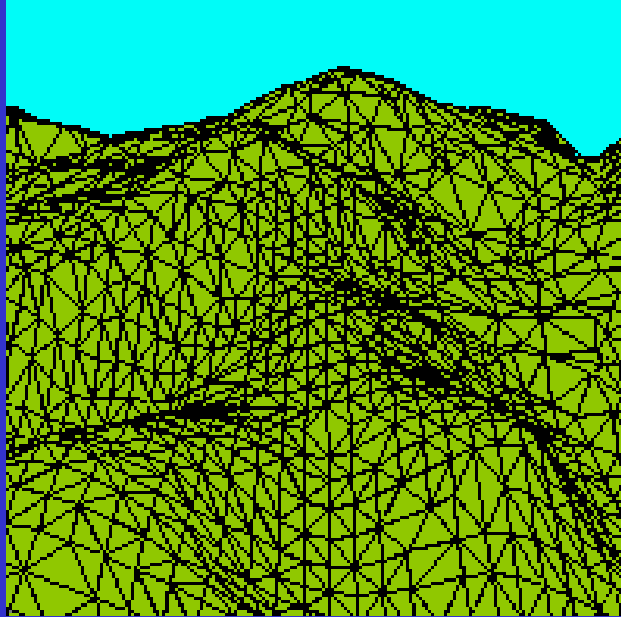
- They can be used for any type of sampled pointset (regularly and irregularly distributed)
- They can adapt to the irregularity of terrains
- They represent continuous surfaces

Triangulated Irregular Networks

The most commonly used polyhedral terrain models are *Triangulated Irregular Networks* (TINs), where each polygon of the domain partition is a triangle



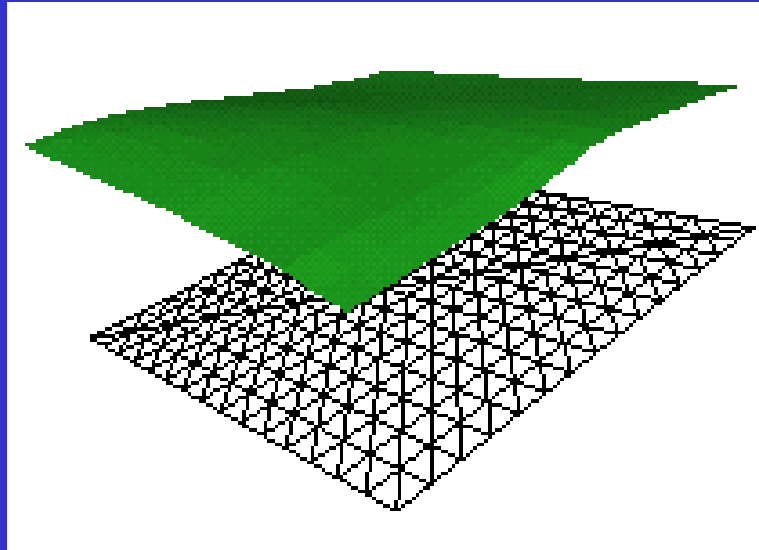
TINs



Example of a TIN based on irregularly distributed data

TINs for regular data

Regular sampling is enough in areas where the terrain elevation is more or less constant



TINs: important properties

They guarantee the existence of a planar patch for each region (triangle) of the domain subdivision (three points define a plane): the resulting surface interpolates all elevation data

The most commonly used triangulations are Delaunay triangulations

Triangular Irregular Network (TIN), which represents a surface as a set of non-overlapping contiguous triangular facets, of irregular size and shape.

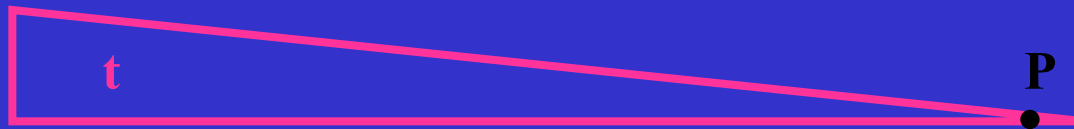
Why Delaunay Triangulations

They generate the most equiangular triangles in the domain subdivision (thus minimising numerical problems: e.g., *point location*)

Their Dual is a Voronoi diagram. Therefore, some proximity queries can be solved efficiently

Delaunay Triangulations

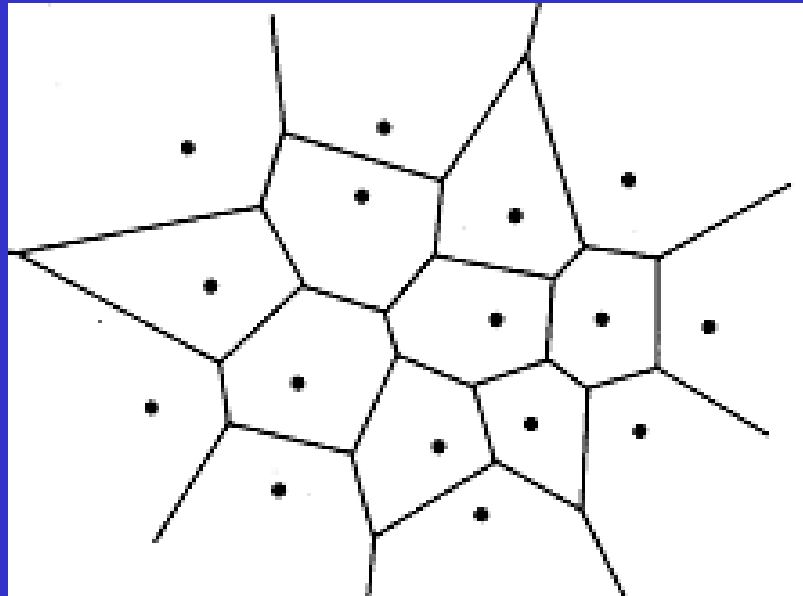
- Intuitively: given a set V of points, among all the triangulations that can be generated with the points of V , the Delaunay triangulation is the one in which triangles are as much equiangular as possible
- In other words, Delaunay triangulations tend to avoid long and thin triangles: important for numerical problems



Does P lie inside t or on its boundary?

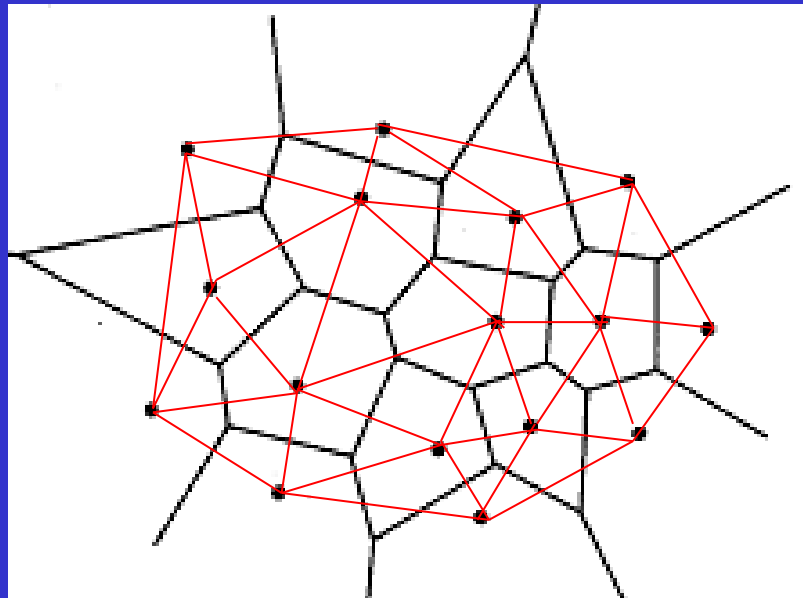
Voronoi Diagrams

- Given a set V of points in the plane, the **Voronoi** Diagram for V is the partition of the plane into polygons such that each polygon contains one point p of V and is composed of all points in the plane that are closer to p than to any other point of V



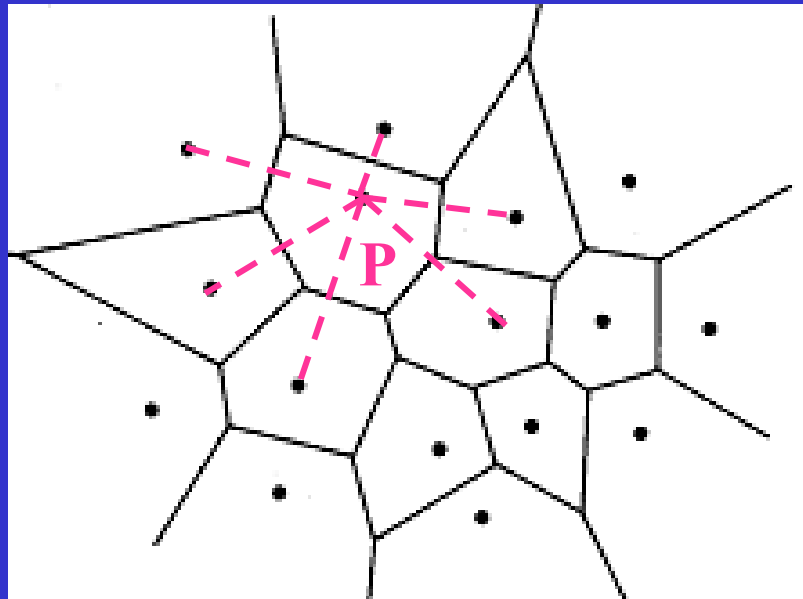
Voronoi Diagrams (cont.d)

- Property: the straight-line **dual** of the Voronoi diagram of V is a Delaunay triangulation of V
- **Dual:** obtained by replacing each polygon with a point and each point with a polygon. Connect all pairs of points contained in Voronoi cells that share an edge



Voronoi Diagrams (cont.d)

- Voronoi diagrams are used as underlying structures to solve *proximity* problems (queries):
 - Nearest neighbour (what is the point of V nearest to P ?)
 - K -nearest neighbours (what are the k points of V nearest to P ?)
 - Etc.



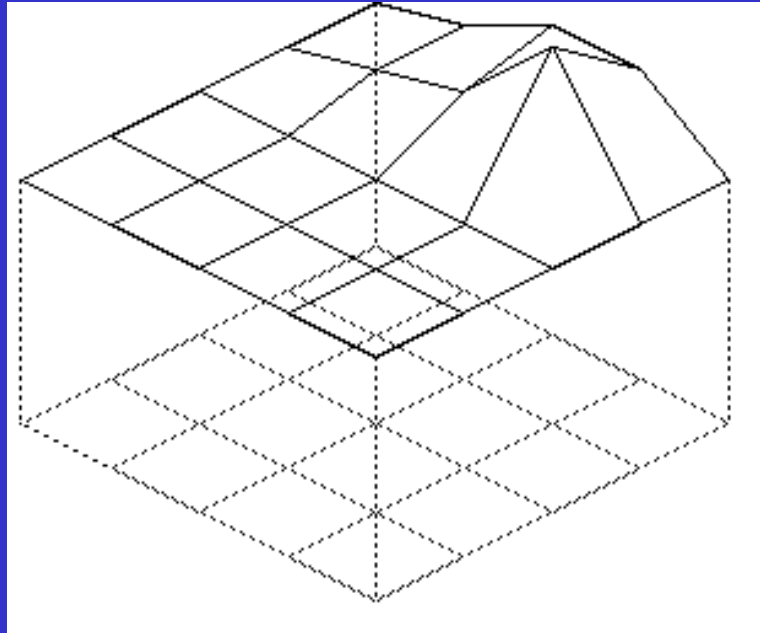
Why Delaunay Triangulations (cont.d)

It has been proven that they generate the best surface approximation (in terms of roughness) independently of the z values

There are several efficient algorithms to calculate them

Gridded models

A Gridded Elevation Model is defined on the basis of a domain partition into regular polygons

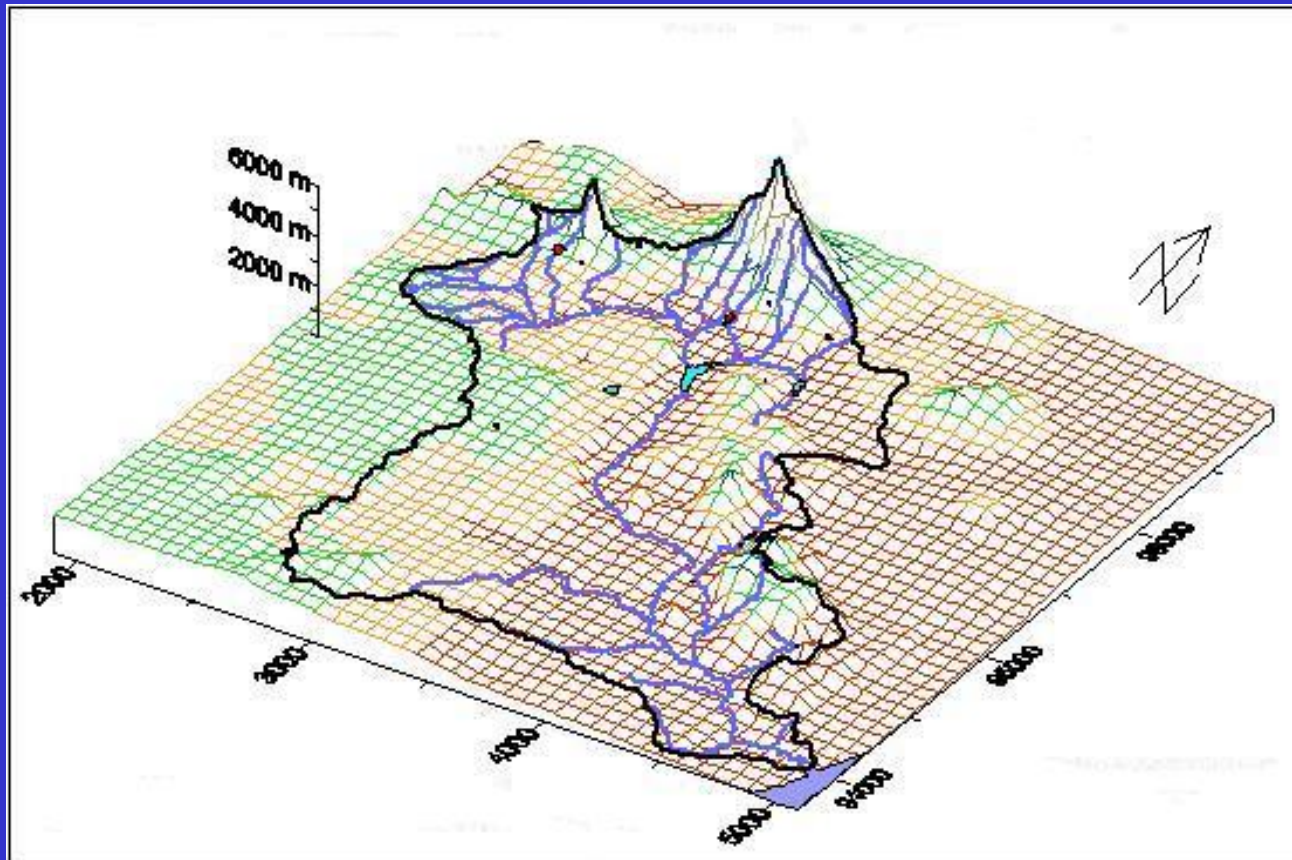


RSGs

The most commonly used gridded elevation models are *Regular Square Grids (RSGs)* where each polygon in the domain partition is a square

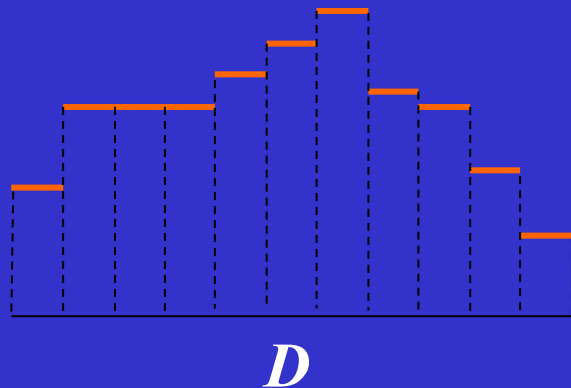
The function defined on each square can be a bilinear function interpolating all four elevation points corresponding to the vertices of the square

RSG: an example



RSG (cont.d)

Alternatively, a constant function can be associated with each square (i.e., a constant elevation value). This is called a *stepped model* (it presents discontinuity steps along the edges of the squares)



Unidimensional
profile of a *stepped
model*

TINs & RSGs

Both models support automated terrain analysis operations

RSGs are based on regular data distribution

TINs can be based both on regular and irregular data distribution

Irregular data distribution allows to adapt to the “variability” of the terrain relief: more appropriate and flexible representation of the topographic surface

Calculations from a Gridded DEM

Slope and Aspect

a	b	c
d	p	e
f	g	h

Consider a 3x3 neighbourhood at every pixel for computation

Slope = Rate of change of terrain property at a given point

Aspect = Direction of Change

Both are expressed as angles

Slope and Aspect

- Slope = S: $\tan S = \left\{ \left(\frac{\partial z}{\partial x} \right)^2 + \left(\frac{\partial z}{\partial y} \right)^2 \right\}^{\frac{1}{2}}$

Where z is the altitude and x and y are the coordinate axes

Aspect: $\tan A = \frac{\partial z / \partial y}{\partial z / \partial x}$

Let $dxx = (a + 2d + f) - (c + 2e + h)$
 $dyy = (a + 2b + c) - (f + 2g + h)$

Slope

$$\frac{\partial z}{\partial x} = \frac{dxx}{(8 * dx)} \quad \text{where } dx = \text{X-resolution}$$

$$\frac{\partial z}{\partial y} = \frac{dyy}{(8 * dy)} \quad \text{where } dy = \text{Y-resolution}$$

Slope

$$S = \arctan \left\{ \left(\frac{\partial z}{\partial x} \right)^2 + \left(\frac{\partial z}{\partial y} \right)^2 \right\}^{\frac{1}{2}}$$

(To convert S into degrees, multiply by 57.29578)

Aspect

- a) If $\frac{\partial z}{\partial x}$ and $\frac{\partial z}{\partial y}$ are positive,

$$\text{Aspect} = 90^\circ + 57.3 * \tan^{-1}[\text{abs}()]$$

- b) If $\frac{\partial z}{\partial x} > 0$ and $\frac{\partial z}{\partial y} < 0$,

- $\text{Aspect} = 90 - 57.3 * \tan^{-1}[\text{abs}()]$

$$\text{Aspect: } \tan A = \frac{\partial z / \partial y}{\partial z / \partial x}$$

Aspect

- c) If $\frac{\partial z}{\partial x} < 0$ and $\frac{\partial z}{\partial y} > 0$,

$$\text{Aspect} = 270 - 57.3 * \tan^{-1} \left[\text{abs} \left(\frac{\partial z / \partial y}{\partial z / \partial x} \right) \right]$$

- d) If $\frac{\partial z}{\partial x} < 0$ and $\frac{\partial z}{\partial y} < 0$,

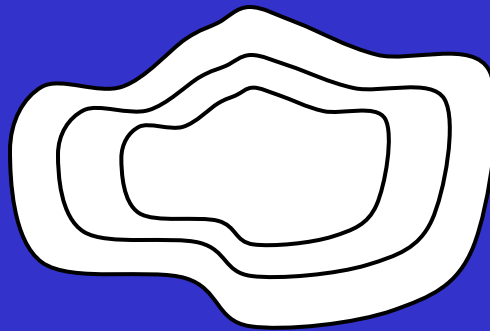
$$\text{Aspect} = 180 - 57.3 * \tan^{-1} \left[\text{abs} \left(\frac{\partial z / \partial y}{\partial z / \partial x} \right) \right]$$

- In all cases, if $\text{Aspect} < 0$, $\text{Aspect} = \text{Aspect} + 360$

Digital Contour Maps

Given a sequence $\{v_0, \dots, v_n\}$ of real values, a *digital contour map* of a mathematical terrain model (D, ϕ) is an approximation of the set of contour lines

$$\{ (x,y) \in D, \phi(x,y) = v_i \} \quad i = 0, \dots, n$$

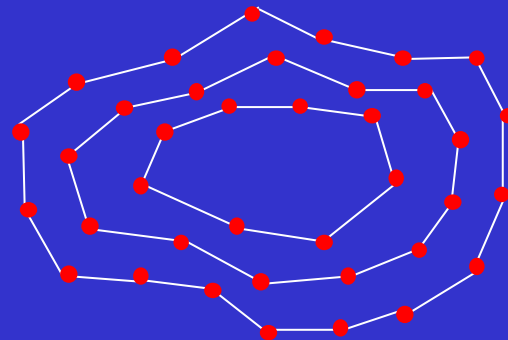
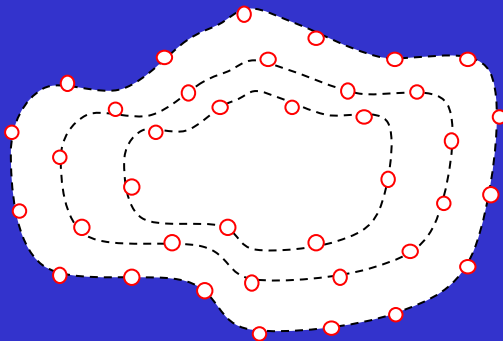


A set of contour
lines

Digital Contour Maps

A line interpolating points of a contour can be obtained in different ways

Examples: **polygonal chains**, or lines described by higher order equations



Digital Contour Maps: properties

They are easily drawn on paper

They are very intuitive for humans

They are not good for complex automated terrain analysis

DTMs: accuracy

In general, the more data is available, the better the representation

Using very large datasets requires large storage space and processing time

Use of generalisation techniques: select a subset of data that still maintains acceptable accuracy in the representation

DTMs: accuracy (cont.d)

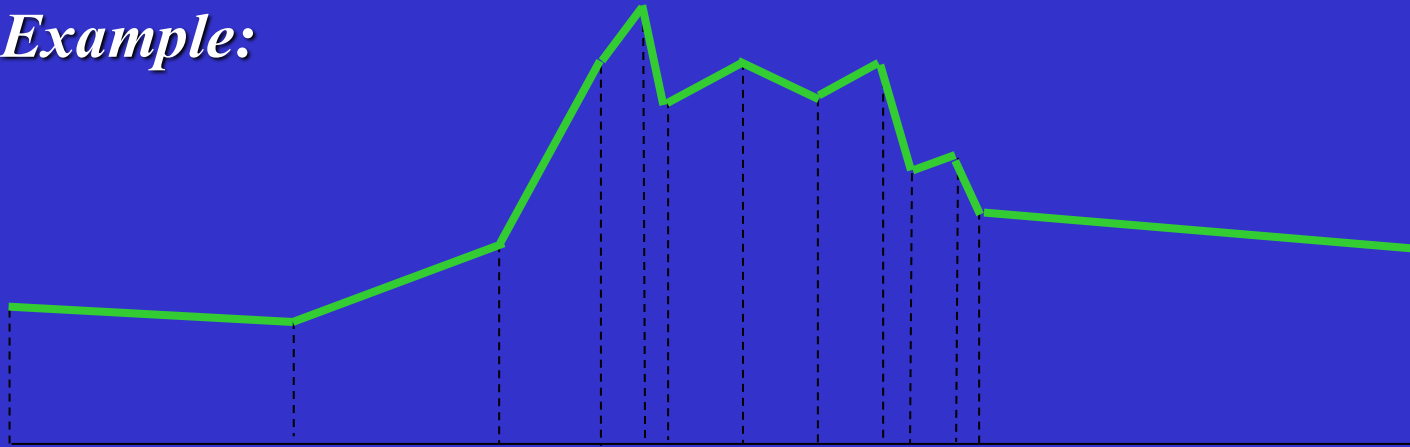
An *approximate terrain* model is a model that uses a subset of the data available

The approximation *error* E is calculated with respect to a reference model built using the whole dataset

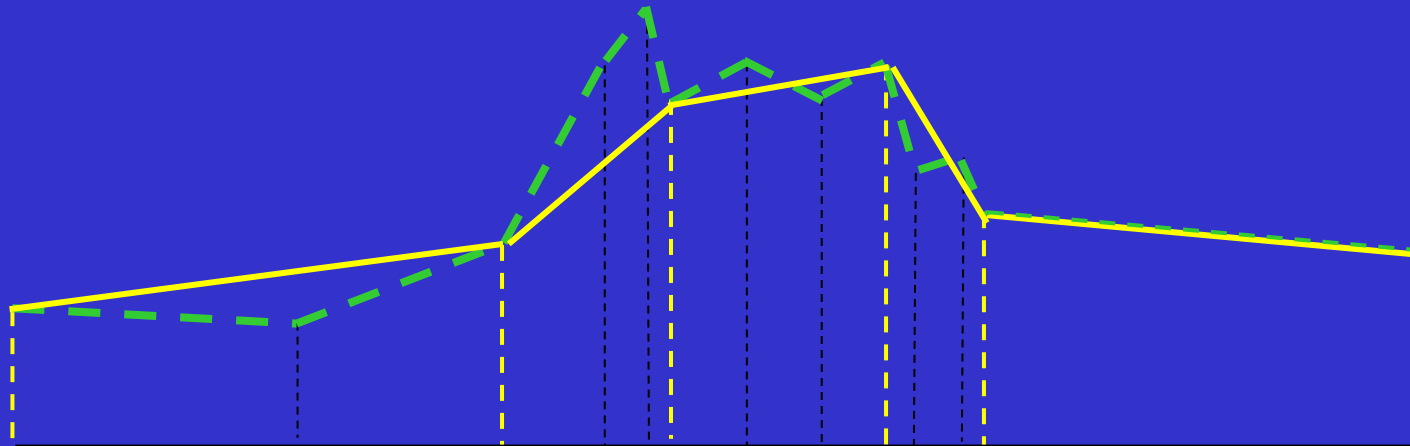
For example: the error could be the maximum difference between the elevation value at a point and the interpolated value in the approximated model

DTMs: accuracy (cont.d)

Example:



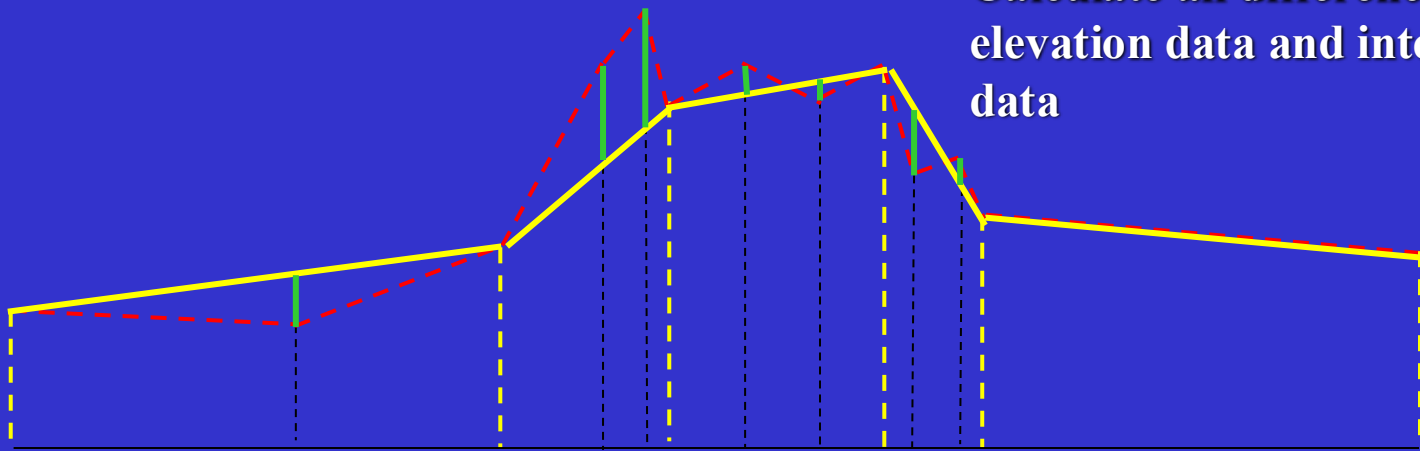
**Model built
using the whole
data set:
*reference model***



**Model built
using a subset of
the data:
*approximated
model***

DTMs: approximation error

Calculate all differences between elevation data and interpolated data



Maximum difference: approximation *error*

DTMs: accuracy (cont.d)

The *accuracy* of the model is inversely proportional to the error E associated with the model and is defined as:

$$\frac{1}{1 + E}$$

Some applications might require to build an approximate model with accuracy (error) within a given threshold

How to pick points

- Given a set of points with known values or given a set of digitized contours, how should points be selected so that the surface is accurately represented?

VIP Algorithm

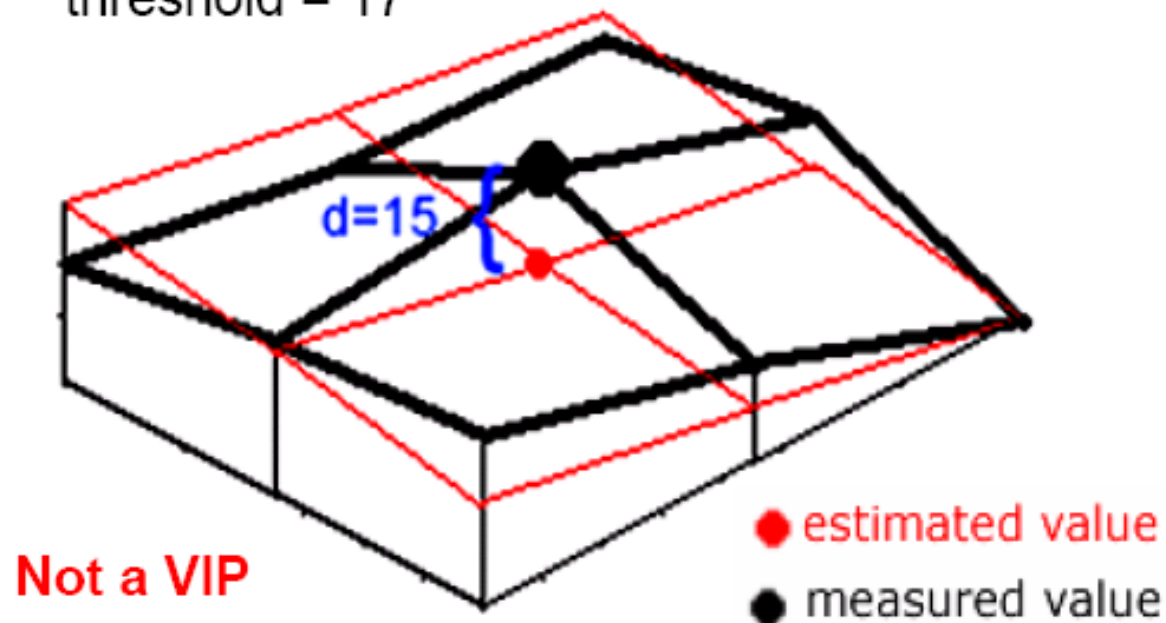
- Each point has 8 neighbors, forming 4 diametrically opposite pairs, i.e. up and down, right and left, upper left and lower right, and upper right and lower left
- For each point, examine each of these pairs of neighbors in turn
 - Connect the two neighbors by a straight line, and compute the perpendicular distance of the central point from this line

VIP Algorithm

- Average the four distances to obtain a measure of "significance" for the point
- Delete points from the DEM in order of increasing significance, deleting the least significant first . This continues until one of two conditions is met: The number of points reaches a predetermined limit . The significance reaches a predetermined limit

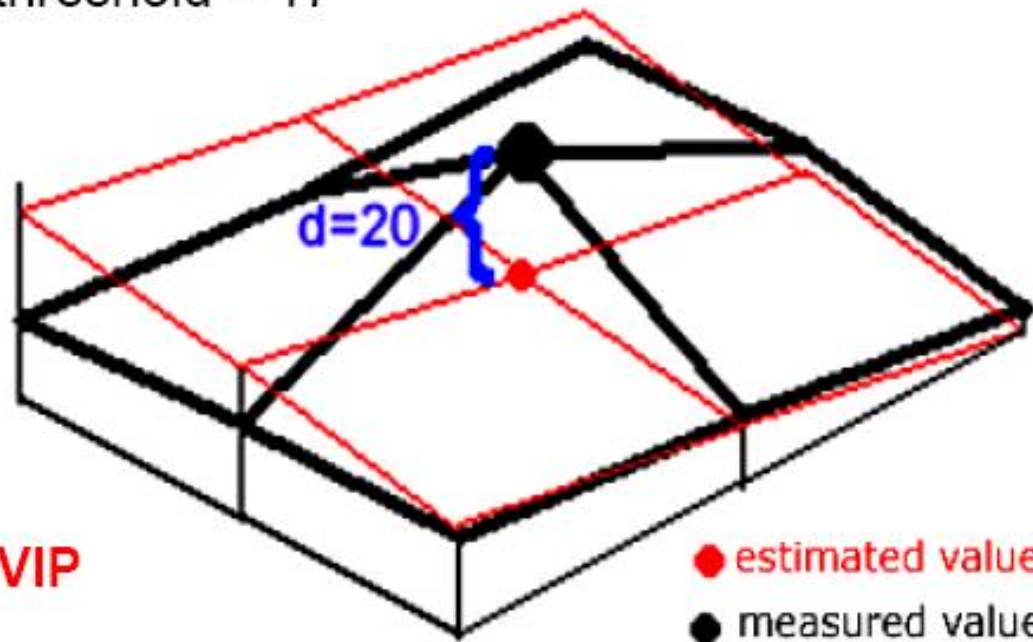
Choosing Data Points-1

- Very Important Points (VIP)
 - threshold = 17



Choosing Data Points-2

- Very Important Points (VIP)
 - threshold = 17

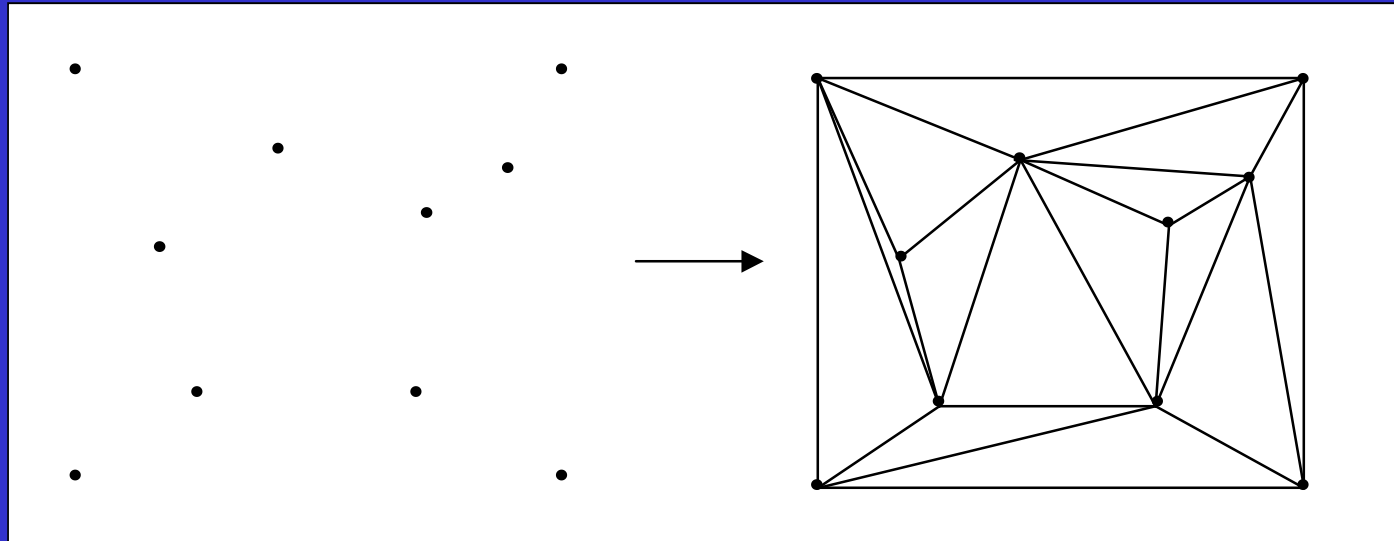


A VIP

Comments

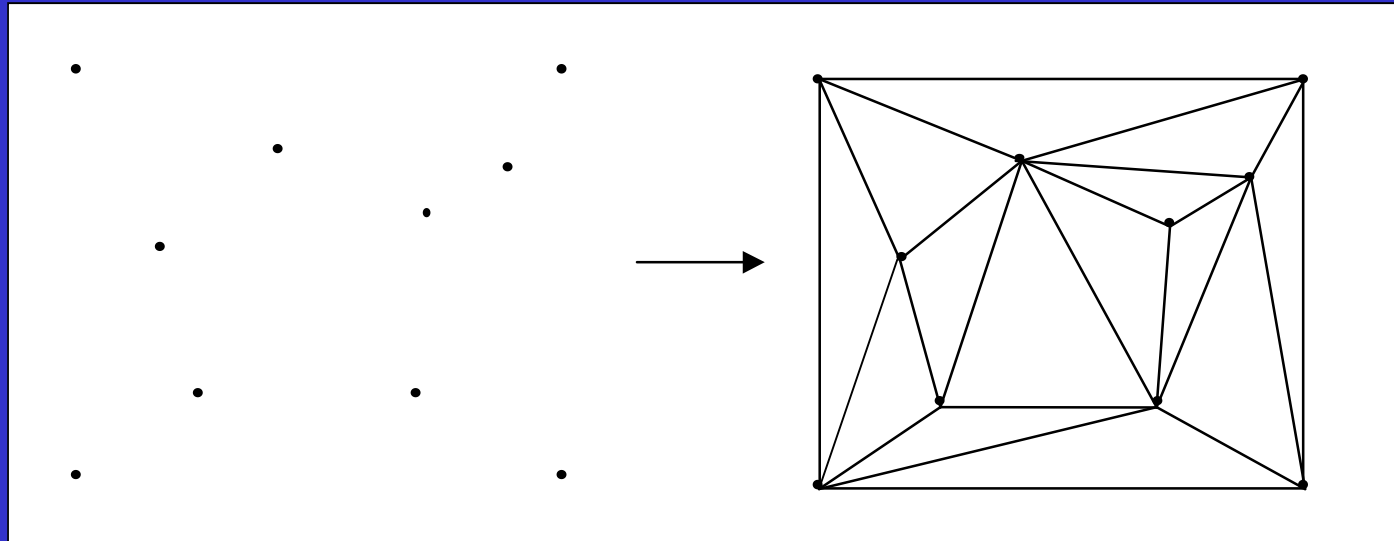
- Due to its local nature, this method is best when the proportion of points deleted is low.
- Due to its emphasis on straight lines, and the TIN's use of planes, it is less satisfactory on curved surfaces.

Triangulation Calculation



- Given a set of points, calculate a triangulation
- Particular properties, e.g., equi-angularity (Delaunay triangulation)

Triangulation Calculation



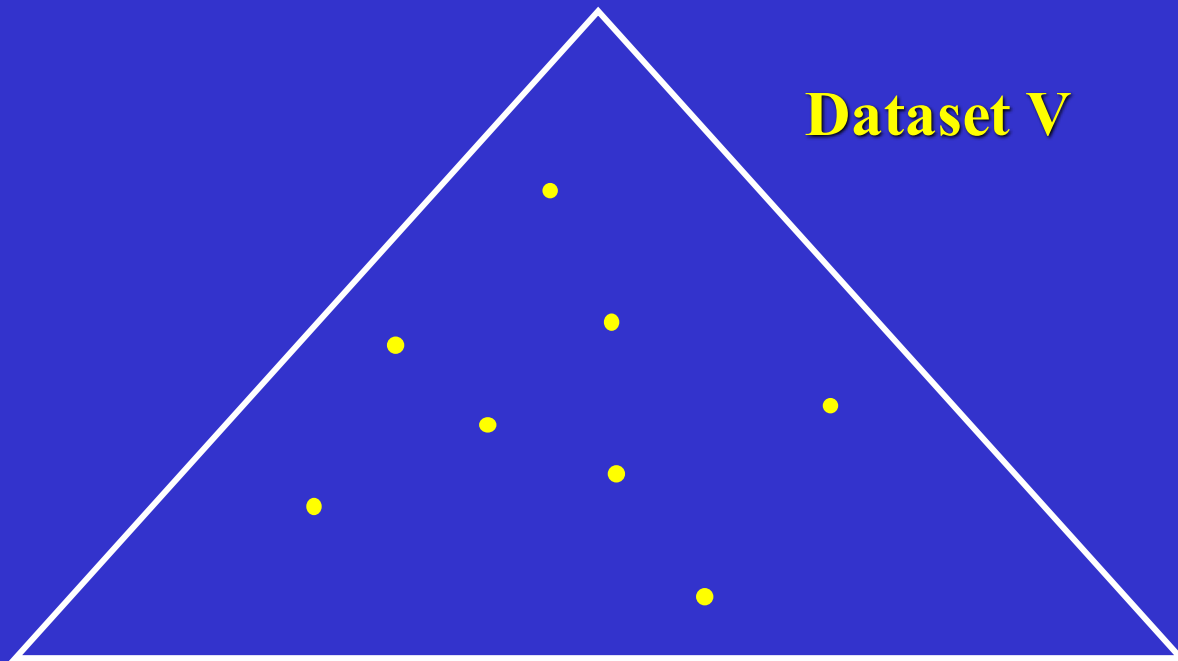
- Given a set of points, calculate a triangulation
- Particular properties, e.g., equiangularity (Delaunay triangulation)

Watson's algorithm (1981)

- Given a set V of points, calculate a Delaunay triangulation with vertices at points of V
- Watson's algorithm is one of the so-called *on-line methods*: based on the modification of an existing Delaunay triangulation when a new point is inserted
- In on-line methods, the first step consists of building a Delaunay triangulation of the domain (containing all data points)
- Then all points of V are added incrementally

Watson: initial step

- In Watson's algorithm, the initial triangulation of the domain is built by considering a fictitious triangle containing all points of V in its interior

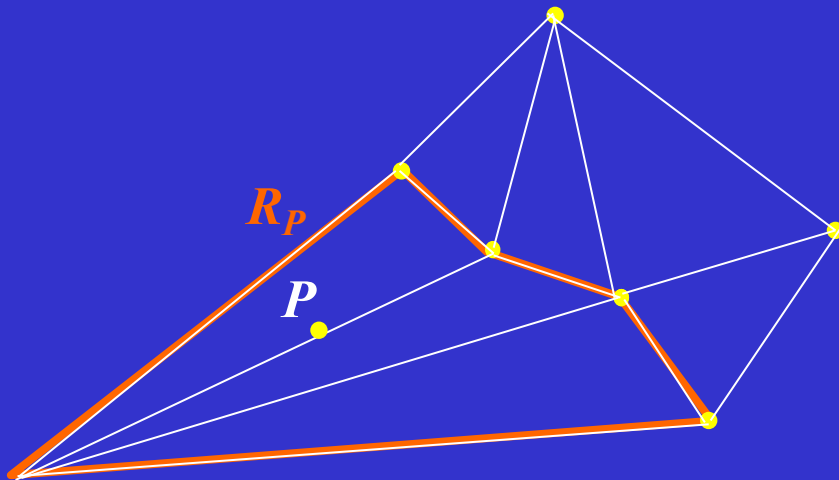


Watson: process

- **After building the initial triangle, all points of V are added one at a time**
- **Finally the initial triangle and all edges incident at its vertices are deleted**
- **The main step in this algorithm is the insertion of a new point in the current Delaunay triangulation**

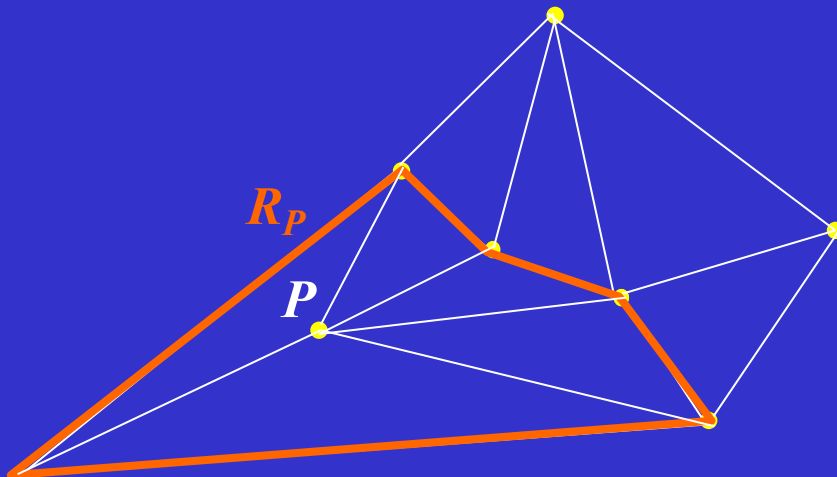
Watson: insertion of a new point

- We call the *influence polygon* R_P of a point P in a triangulation T the union of all triangles of T whose circumscribing circle contain P
- After inserting P in T , we update T by deleting all edges internal to R_P and by joining P with all the vertices of R_P



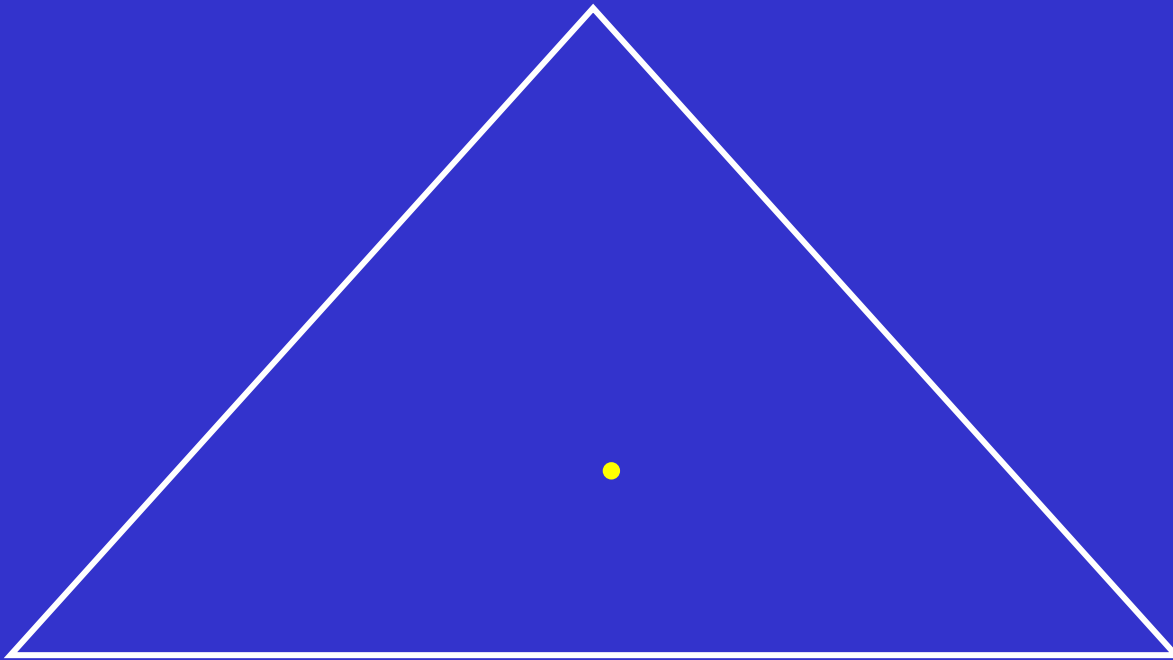
Watson: insertion of a new point

- We call the *influence polygon* R_P of a point P in a triangulation T the union of all triangles of T whose circumscribing circle contains P
- After inserting P in T , we update T by deleting all edges internal to R_P and by joining P with all the vertices of R_P



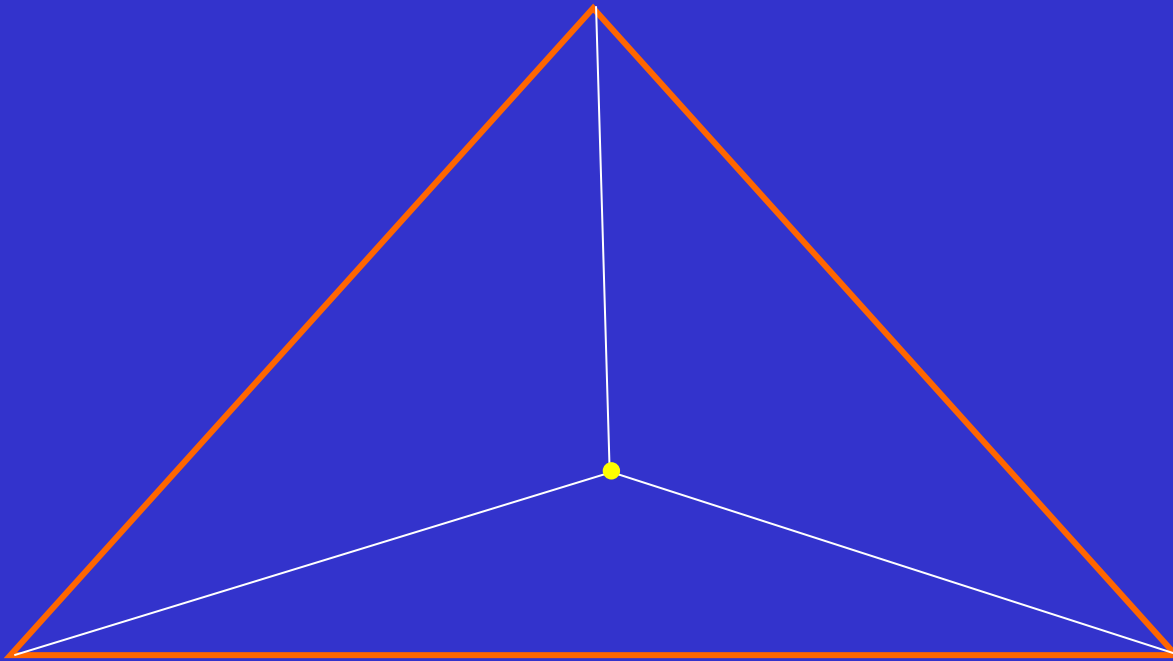
Watson: insertion step

- Points in V are added one at a time in the current Delaunay triangulation



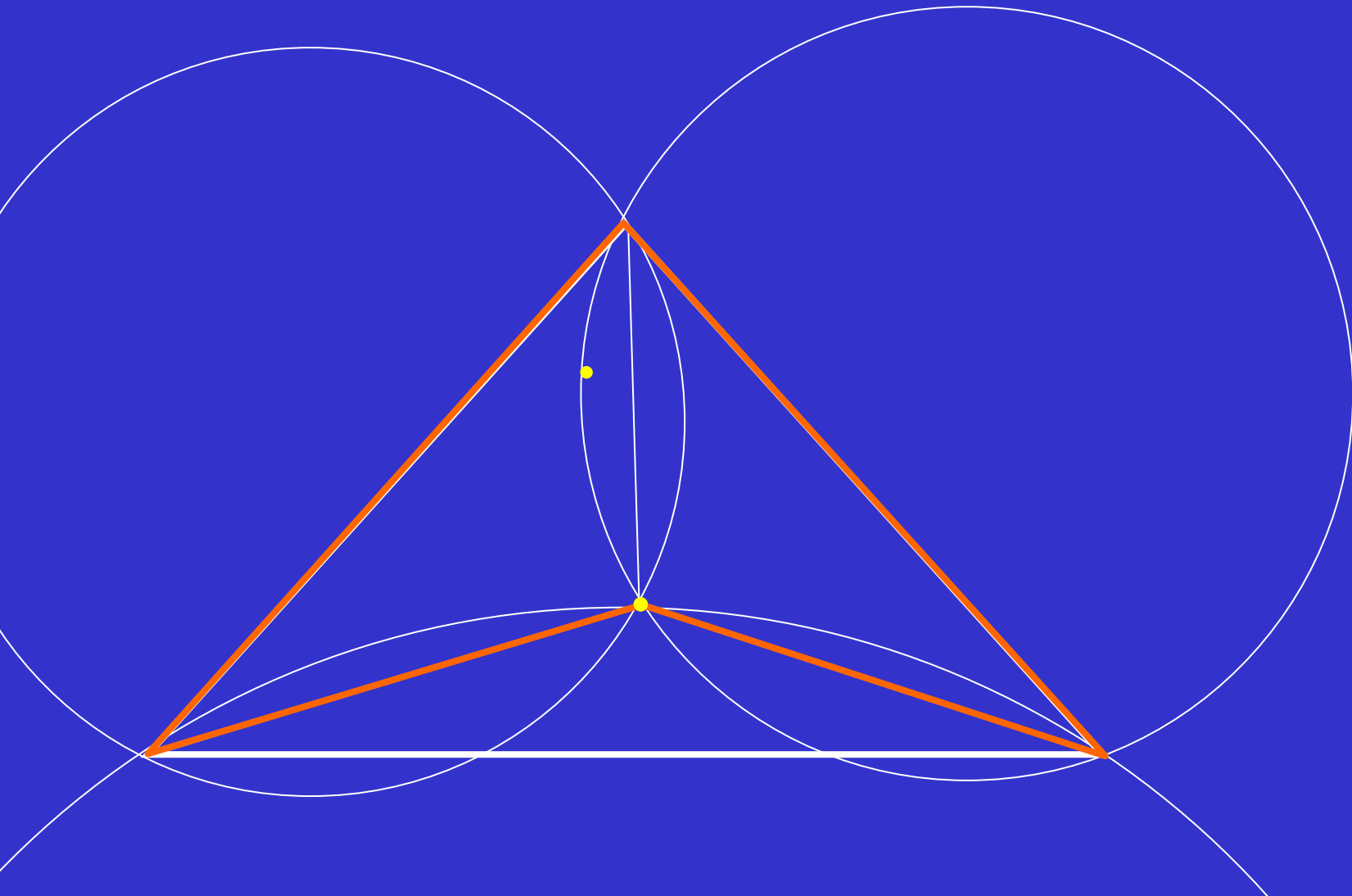
Watson: insertion step (cont.d)

- First vertex P_I : the fictitious triangle is its influence polygon R_{P_I} ; join P_I with its three vertices



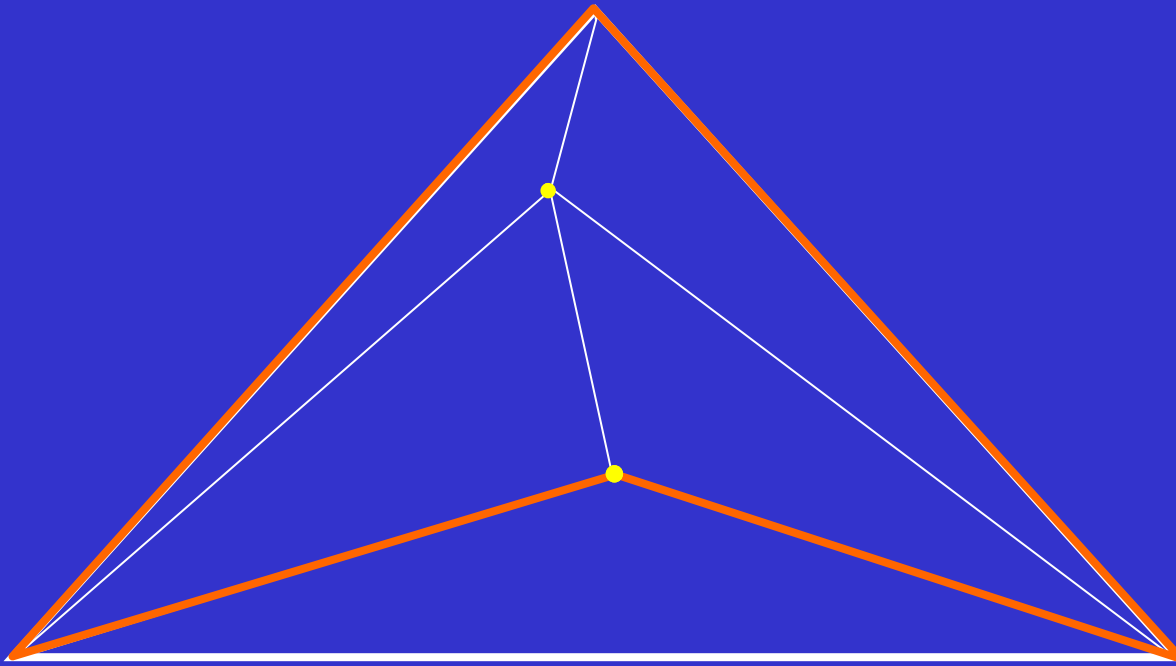
Watson: insertion step (cont.d)

- Inserting the second vertex P_2 : calculation of R_{P1}



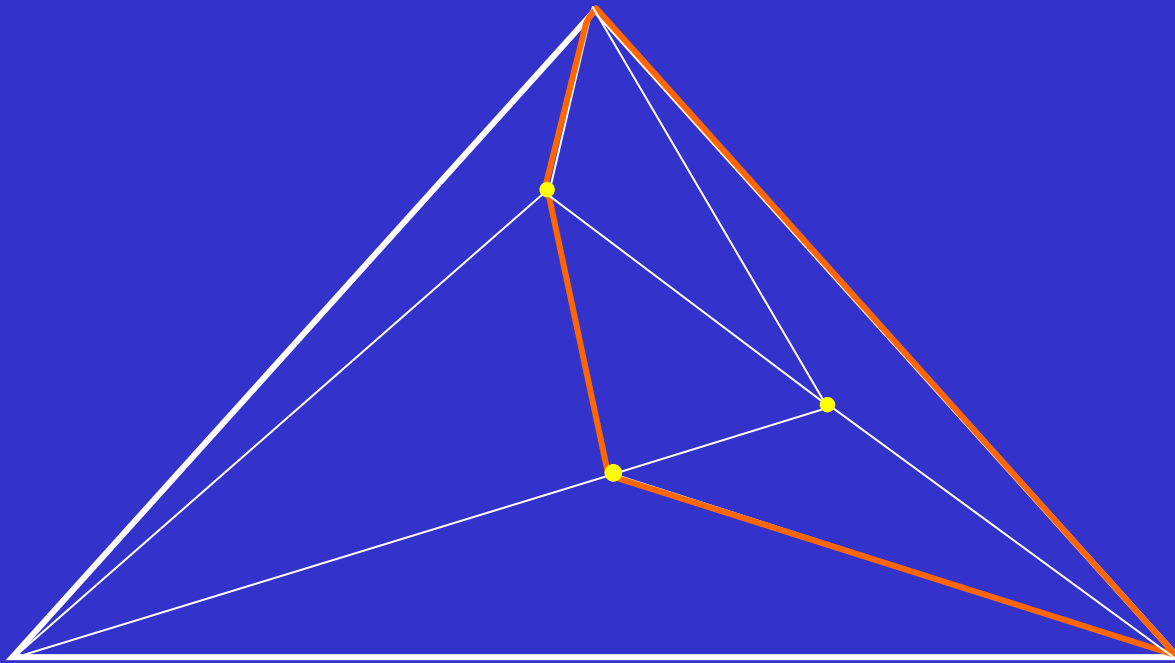
Watson: insertion step (cont.d)

- Inserting the second vertex P_2 : updating the current triangulation



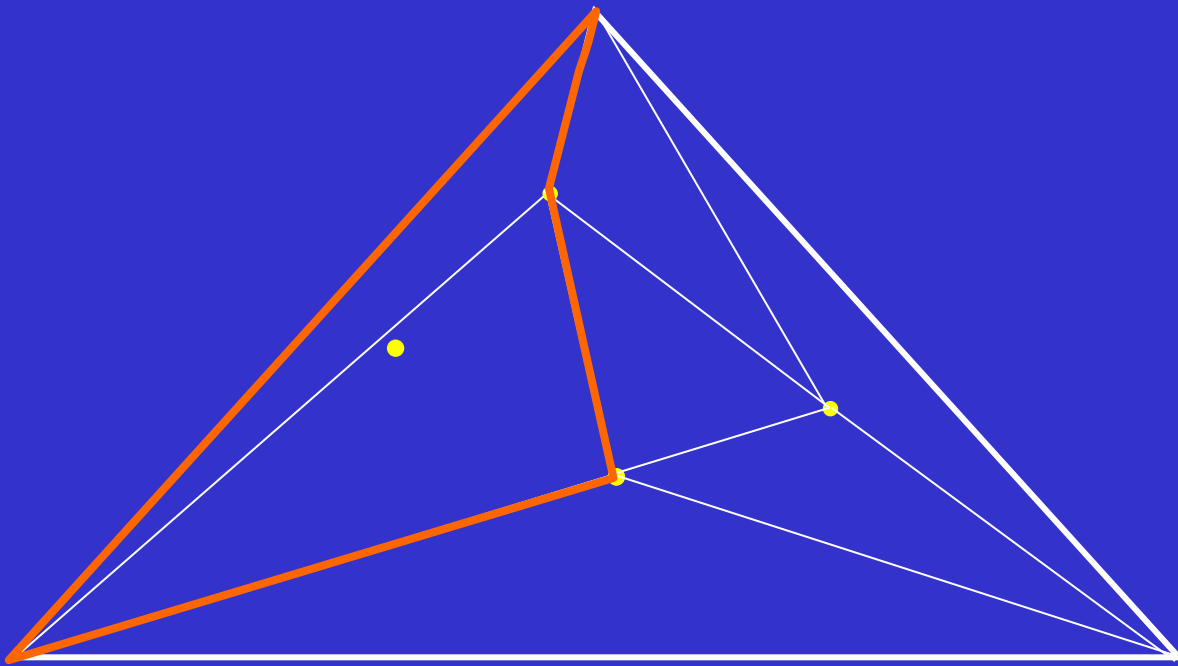
Watson: insertion step (cont.d)

- Inserting new vertices



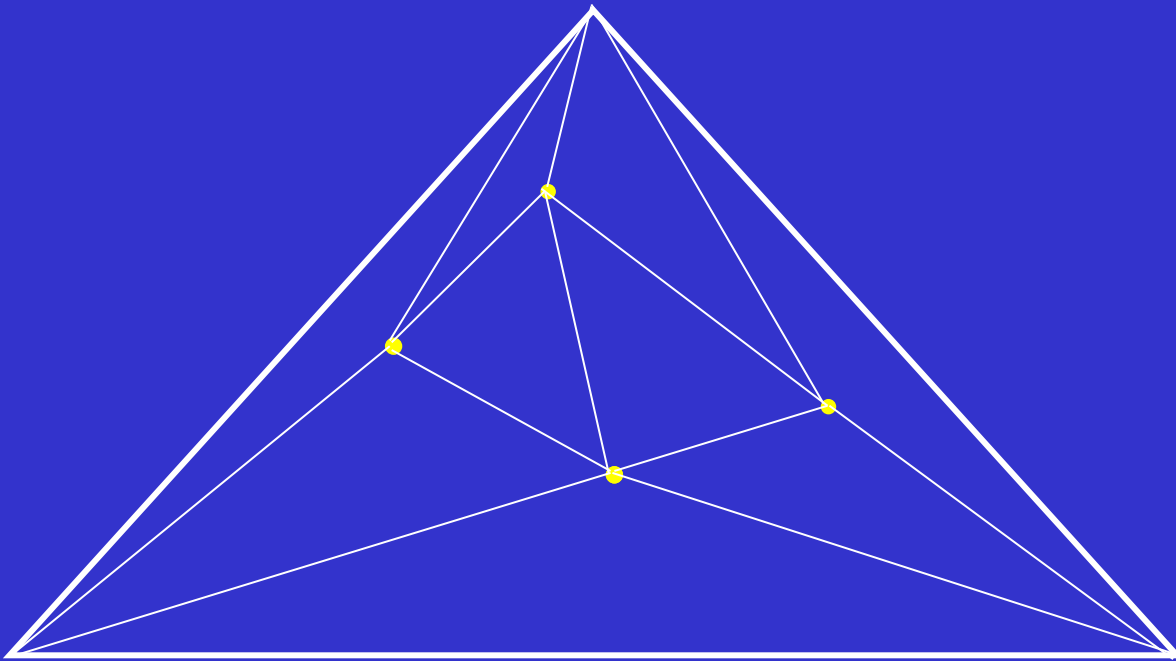
Watson: insertion step (cont.d)

- Inserting new vertices



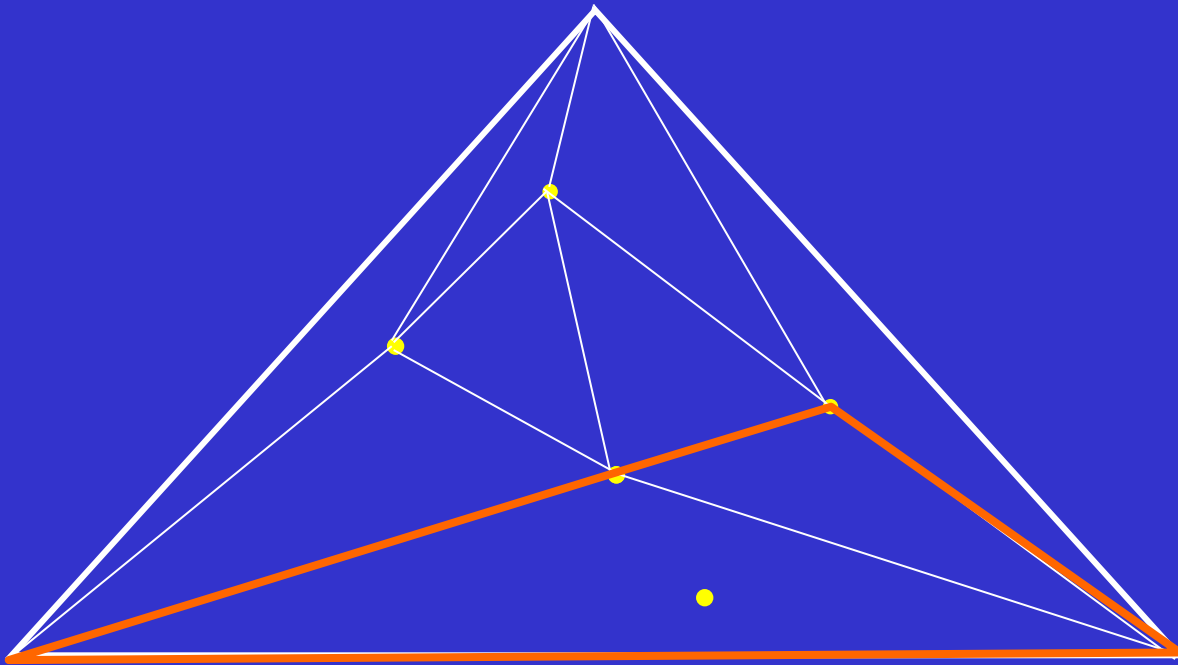
Watson: insertion step (cont.d)

- Inserting new vertices



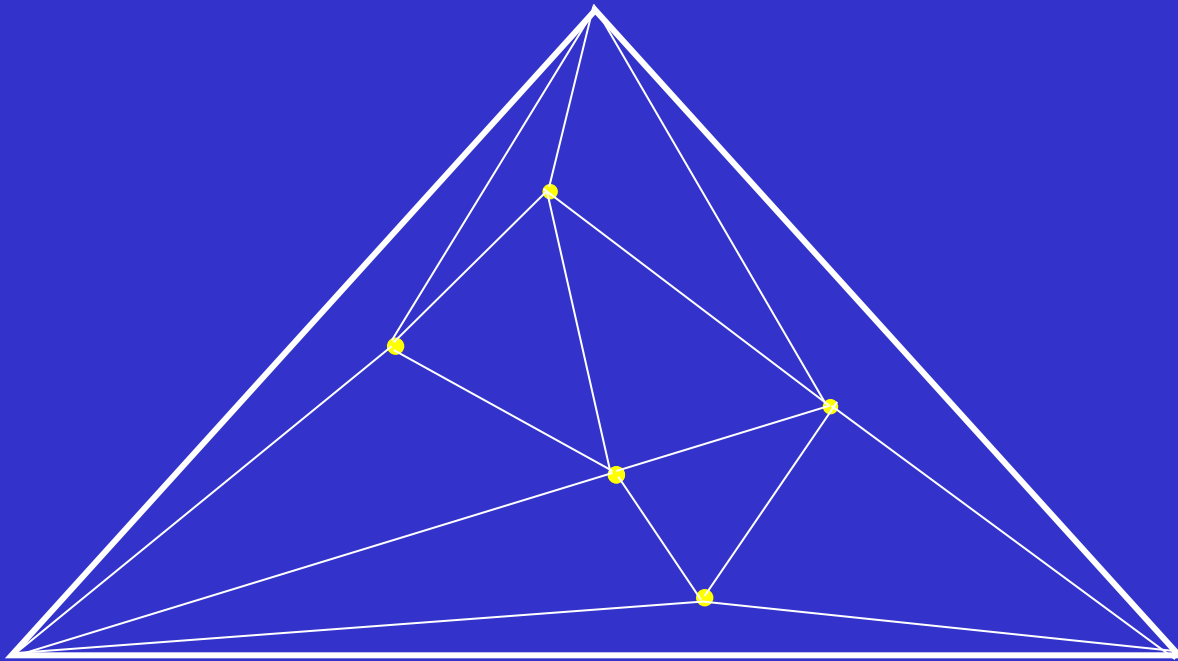
Watson: insertion step (cont.d)

- Inserting new vertices



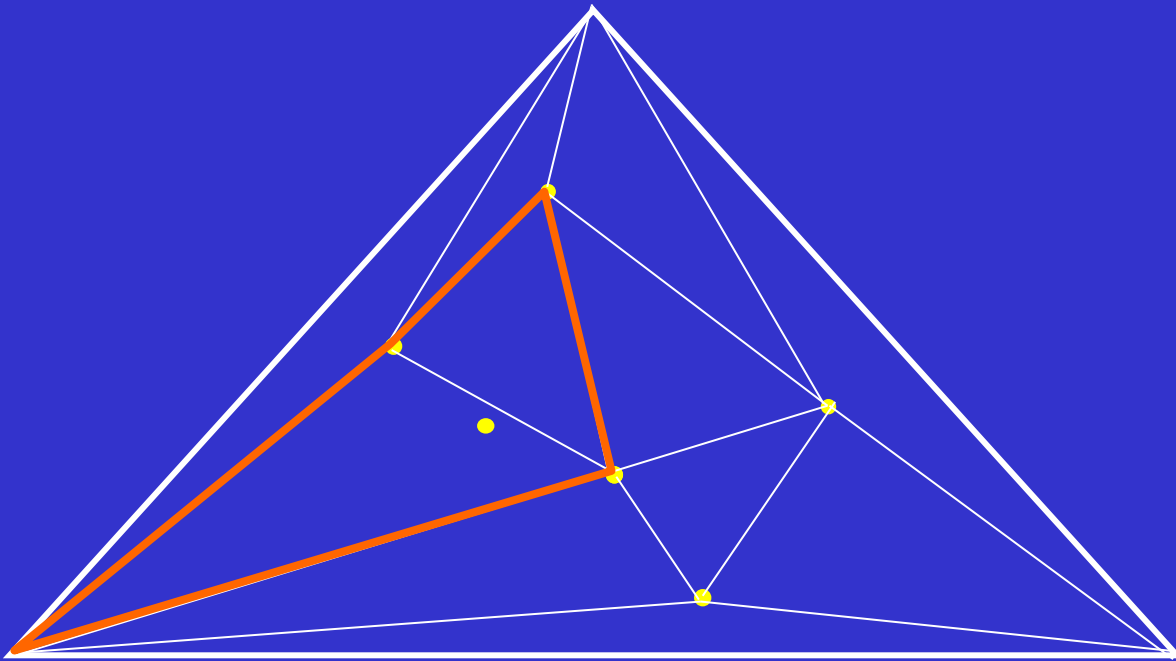
Watson: insertion step (cont.d)

- Inserting new vertices



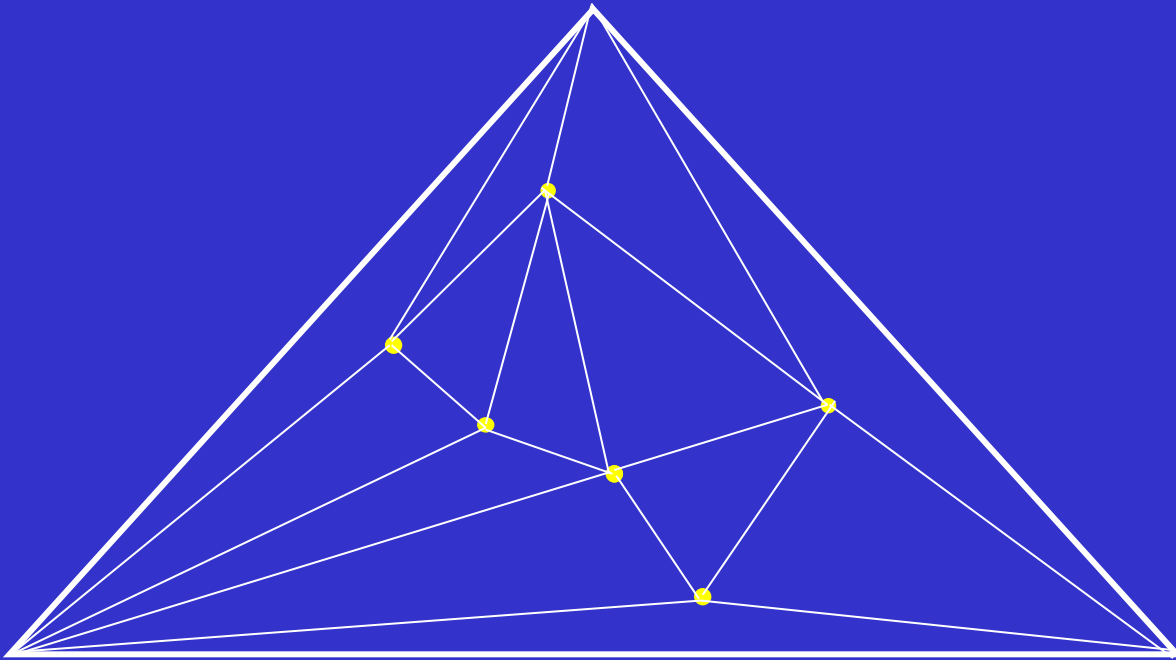
Watson: insertion step (cont.d)

- Inserting new vertices



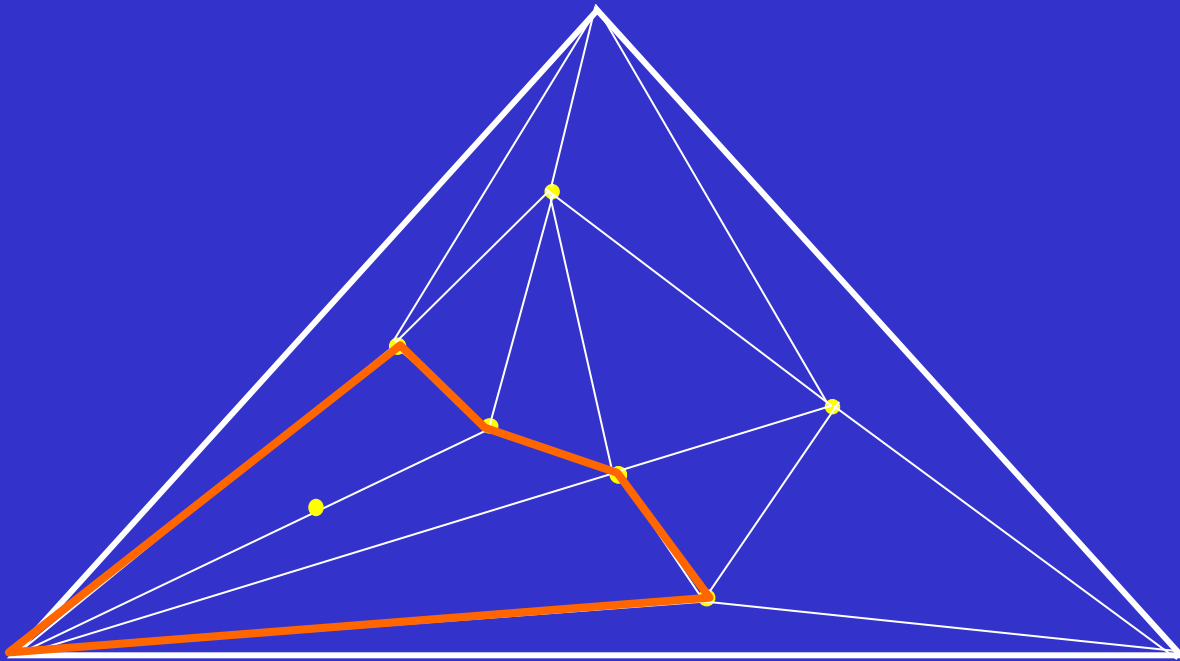
Watson: insertion step (cont.d)

- Inserting new vertices



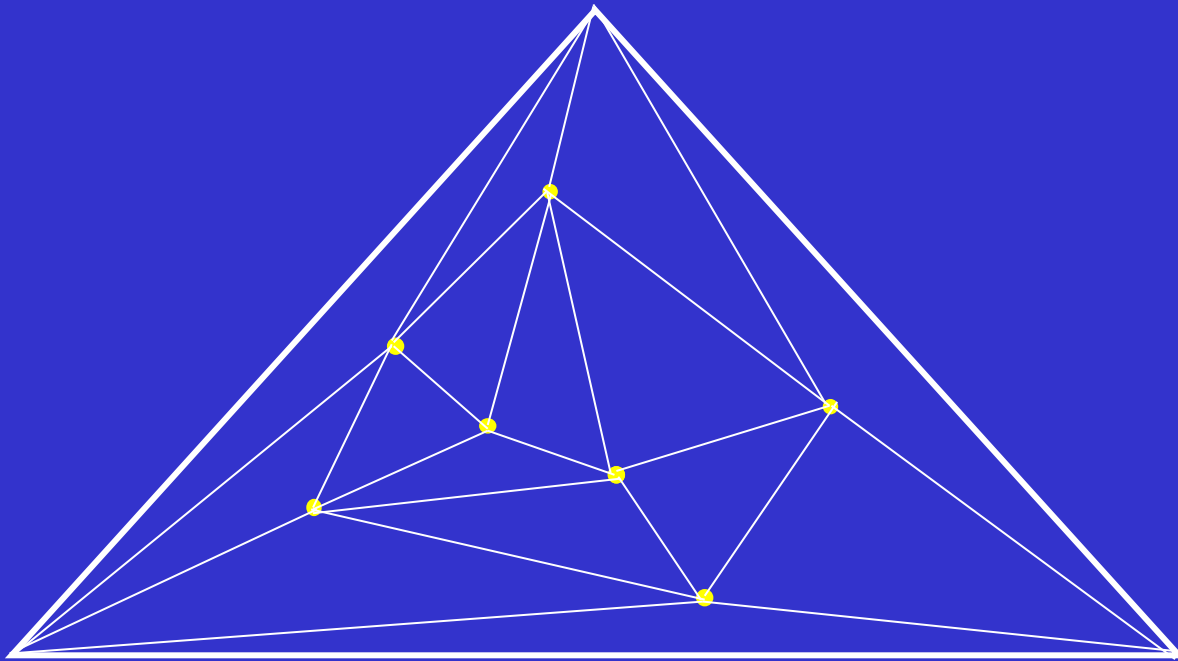
Watson: insertion step (cont.d)

- Inserting new vertices



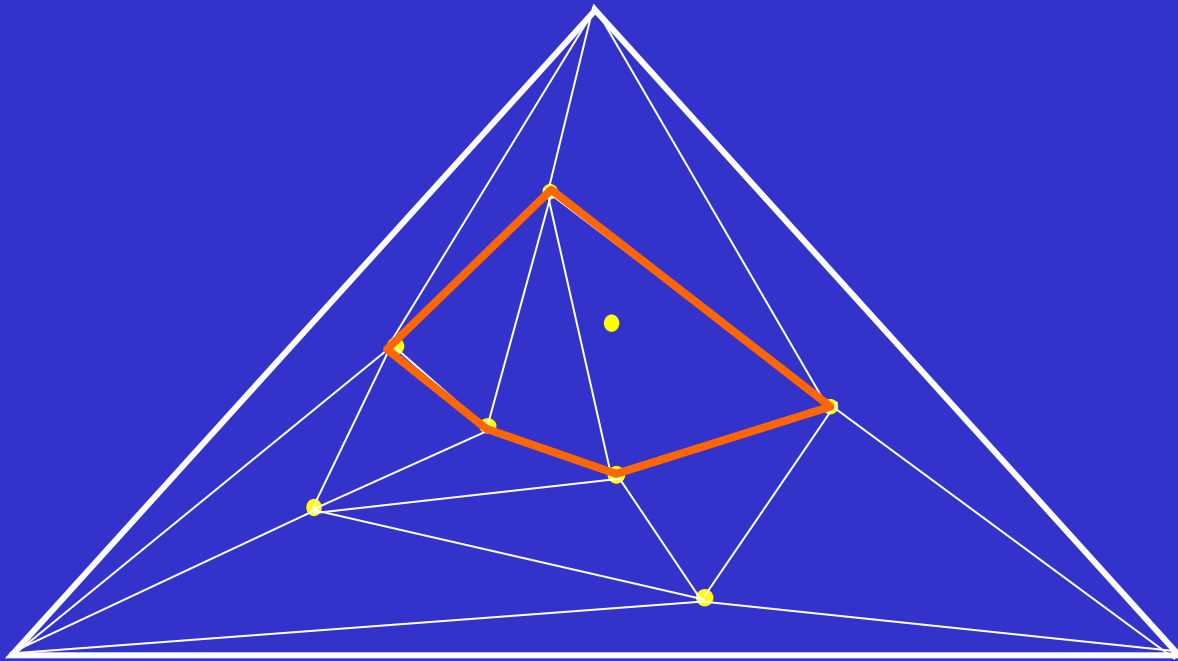
Watson: insertion step (cont.d)

- Inserting new vertices



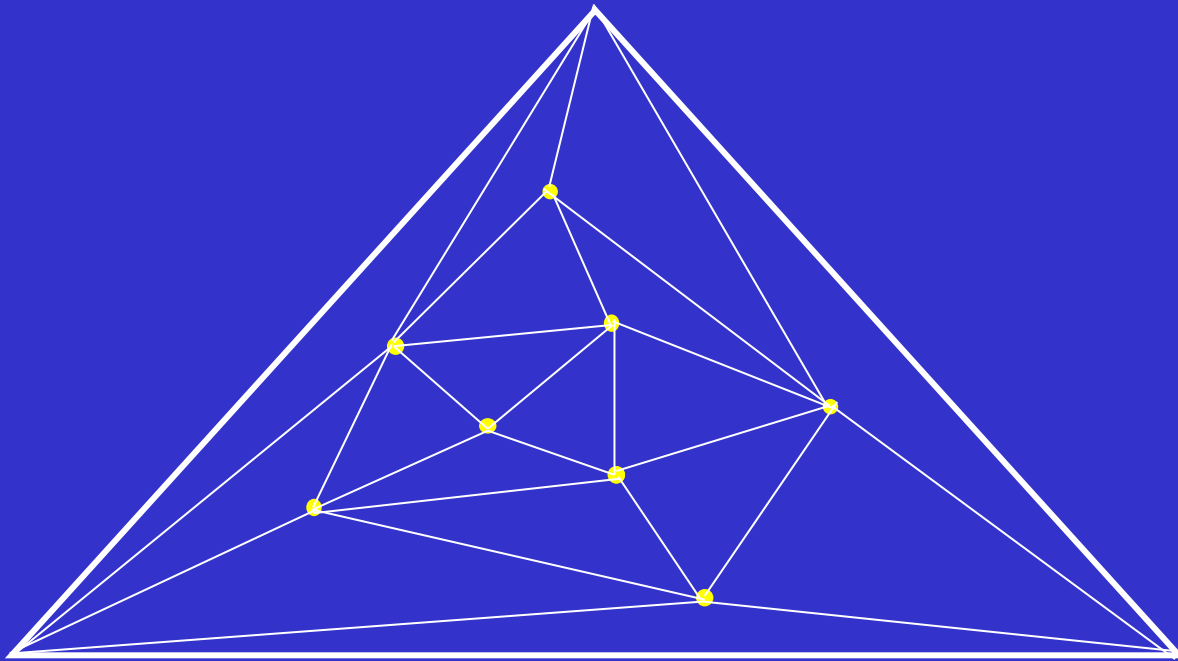
Watson: insertion step (cont.d)

- Inserting new vertices



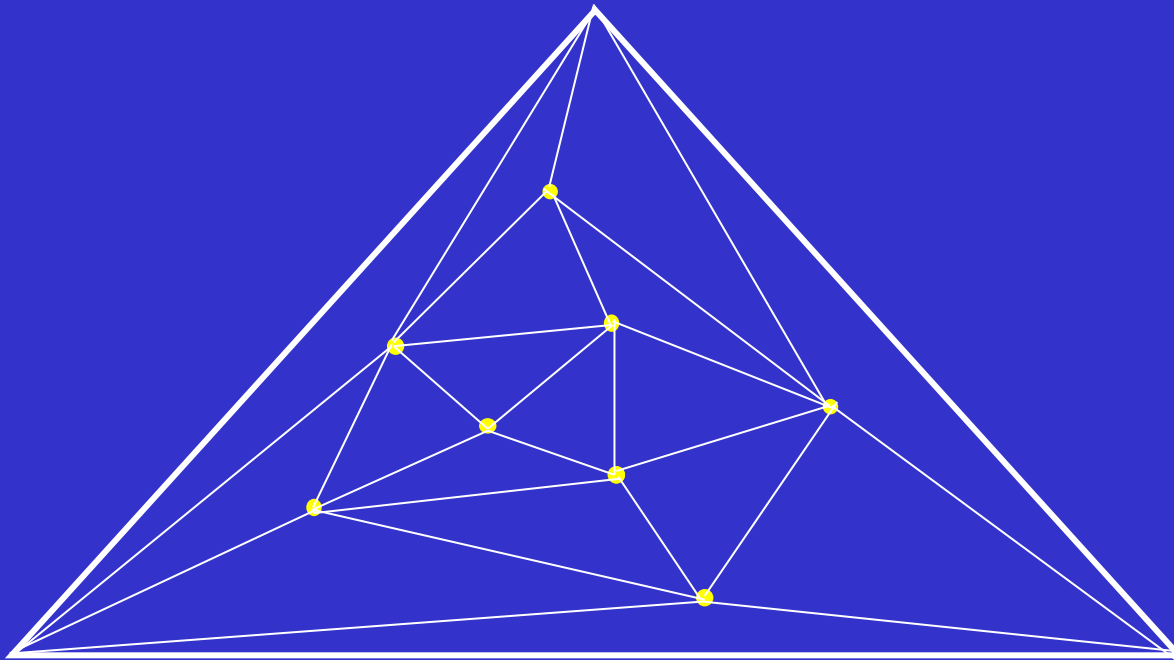
Watson: insertion step (cont.d)

- Inserting new vertices



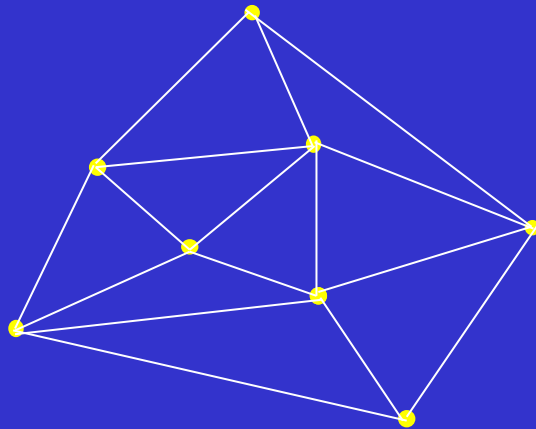
Watson: final step

- Delete the fictitious triangle and all edges incident at its vertices



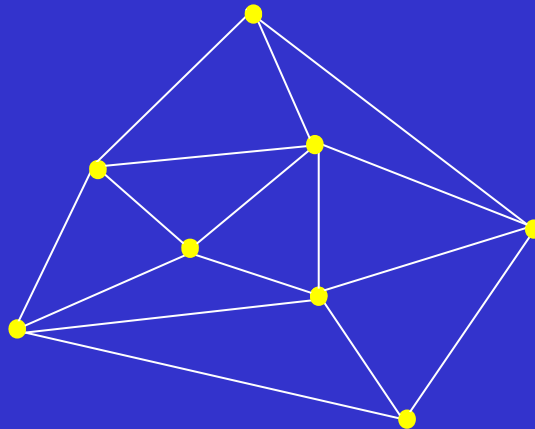
Watson: final step

- Delete the fictitious triangle and all edges incident at its vertices



Watson: final step

- Given V , Watson's algorithm calculates the Delaunay triangulation with vertices at points of V



Watson: complexity

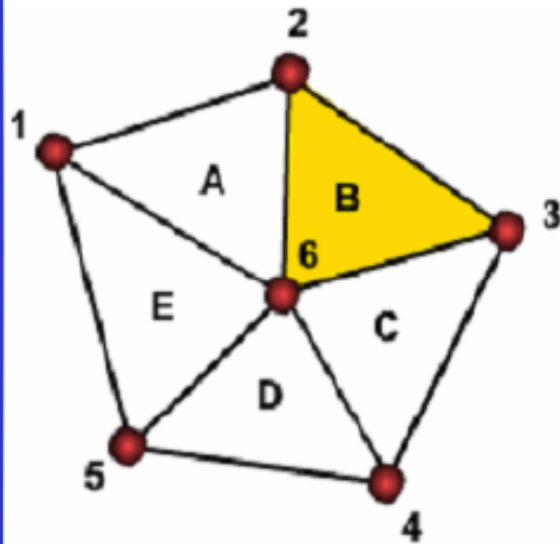
- The time complexity of Watson's algorithm is $O(n^2)$ where n is the number of input points (worst case)
- This depends on the fact that the insertion of a new point requires changing all triangles of the influence polygon. In the worst case, the influence polygon includes all triangles of the current triangulation
- NOTE: remember that the number of triangles in a triangulation is $O(v)$, with v the number of vertices in the triangulation

Watson: complexity (cont.d)

- Although the worst case time complexity for Watson's algorithm is $O(n^2)$, it has been shown that in the average case the number of triangles in the influence polygon is equal to 6
- Therefore, the average case time complexity is linear in the number of input points

Data Structure-1

Triangle Table



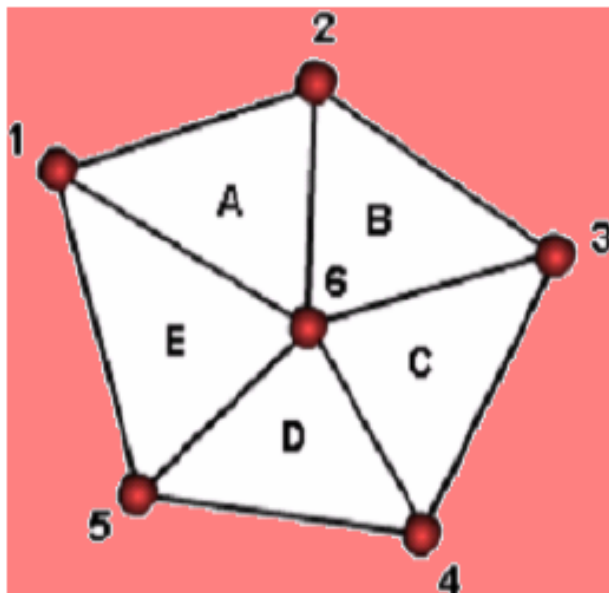
face	P1	P2	P3	N1	N2	N3
A	1	2	6	0	B	E
B	2	3	6	0	C	A
C	6	3	4	B	0	D
D	6	4	5	E	C	0
E	1	6	5	A	D	0

node	X	Y	Z
1	...		
2			
3			
4			
5			
6			

Storage of Triangle Data

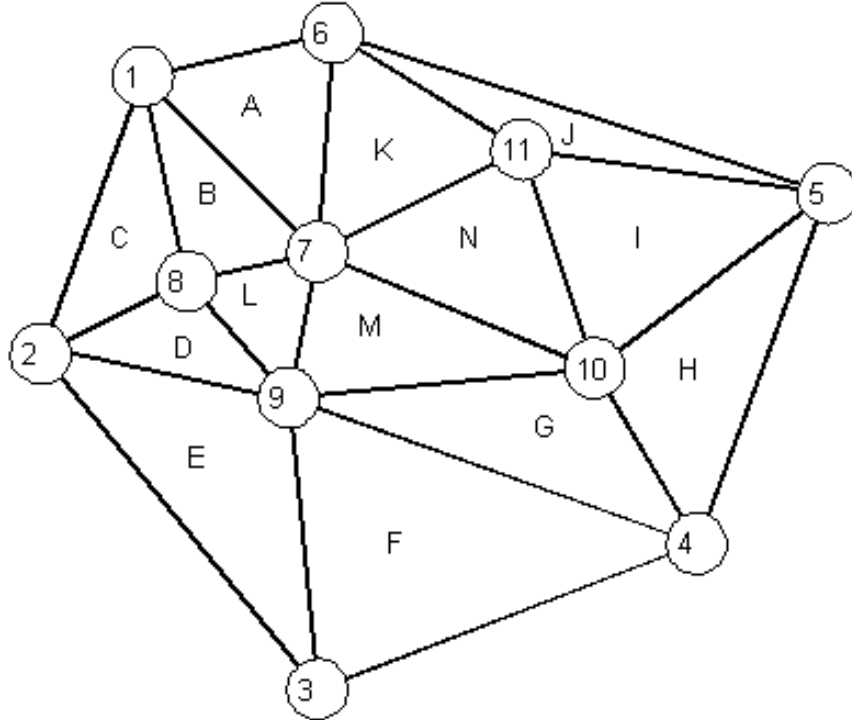
Data Structure-2

Node Table



node	x	y	z	edges to
1	...	...	...	2 6 5
2				3 6 1
3				4 6 2
4				3 5 6
5				6 4 1
6				2 3 4 5 1

Storage of Triangle Data



EDGES		
△	adjacent	△
A	B, K	
B	A, C, L	
C	B, D	
D	C, E, L	
E	D, F	
F	E, G	
G	F, H, M	
H	G, I	
I	H, J, N	
J	I, K	
K	A, J, N	
L	B, D, M	
M	G, L, N	
N	I, K, M	

NODES	
△	node#
A	1, 6, 7
B	1, 7, 8
C	1, 2, 8
D	2, 8, 9
E	2, 3, 9
F	3, 4, 9
G	4, 9, 10
H	4, 5, 10
I	5, 10, 11
J	5, 6, 11
K	6, 7, 11
L	7, 8, 9
M	7, 9, 10
N	7, 10, 11

X-Y Coordinates	
node#	coordinates
1	x1, y1
2	x2, y2
3	x3, y3
...	...
11	x11, y11

Z Coordinates	
node#	z_value
1	z1
2	z2
3	z3
...	...
11	z11

The structure of a TIN. The TIN is a topological data model. The data are stored in a set of tables that retain the coordinate values as well as the spatial relations of the facets.

TIN Storage

- Performance of any structure based operation, e.g., algorithm development depends on the way the data are stored.
- In TIN based DTM, data can be stored as
 - Triangle-based (for slope, aspect, volume computations)
 - Node-based
 - Side-based (for contouring or any traversal procedure)
 - Combination of the above (for visibility analysis)
- TIN structure is a case of establishing a good and efficient triangle topology

TIN Applications

- Slope and aspect
- Contouring
- Finding drainage networks
- Creating cross-sections
- Visualization of a 3-dimensional surface